



GitHub Certified: Agentic AI Developer

GH-600 Beta — Practice Exam

68 questions · Friday, May 22, 2026

Correct answers highlighted · Full explanations included

Questions & Answers

Q1**Domain A: Agent architecture and SDLC integration****sequence order****very_hard**

A payments platform team wants Copilot cloud agent to handle low-risk developer-experience fixes, but recent agent runs mixed exploration, implementation, and review into one opaque session. The team redesigns the workflow so each run produces reviewable evidence before moving to the next stage. In what order should the workflow steps occur?

The goal is to preserve delivery speed while making planning distinct from execution and using GitHub PRs, comments, and checks as the standard system of record.

Correct order:

1. Capture the task contract from the issue: scope, inputs, constraints, and success criteria.
2. Require the agent to publish a structured implementation plan as a comment or draft PR section.
3. Validate the plan against the task contract and resolve missing assumptions before implementation begins.
4. Allow the agent to create commits on its branch and update the pull request description.
5. Run automated checks and request review so the output is evaluated through the PR workflow.
6. Have a human merge after reviewing the PR diff, plan alignment, and completed checks.

✓ **Why the correct answer is right**

The correct sequence starts with a defined task contract, then produces a structured plan, validates that plan before action, performs implementation on an agent branch, evaluates the result with checks and review, and finishes with a human merge decision. This preserves the plan, act, and evaluate lifecycle while keeping GitHub as the inspectable control plane.

✗ **Why the other options are wrong**

- **A:** Merging belongs at the end because it depends on reviewed changes and completed evaluation signals.
- **C:** Implementation should not precede a structured plan and validation of assumptions against the issue contract.
- **D:** Planning should occur after the task scope is known, not before inputs and success criteria are captured.
- **E:** Checks and review evaluate the agent output after commits exist, not before implementation starts.
- **F:** Plan validation requires a plan and the original task contract, so it cannot be first.

💡 Sequence questions in this domain usually reward explicit stage boundaries. Put task definition before planning, plan validation before action, and evaluation before merge.

Q2

Domain A: Agent architecture and SDLC integration

multi select

hard

A team reviews several Copilot agent sessions and notices two recurring patterns: failures are described only as vague final summaries, and reviewers cannot tell which validation steps actually ran. Select TWO workflow changes that directly mitigate these SDLC anti-patterns.

The team wants reviewers to detect agent failure modes earlier while keeping GitHub issues, pull requests, and session logs as the primary artifacts.

- A** Require unrecoverable setup or test failures to be surfaced as explicit PR comments or checklist failures. ✓
- B** Ask reviewers to inspect only the final diff because logs may contain too much detail.
- C** Require the agent to record attempted validation steps and outcomes in the pull request. ✓
- D** Increase the maximum issue description length so users can include more background context.
- E** Route all low-risk agent tasks through a weekly manual planning meeting before assignment.

✓ **Why the correct answer is right**

B creates a structured validation artifact reviewers can inspect. E prevents silent failures by making unrecoverable errors visible in standard GitHub review surfaces.

✗ **Why the other options are wrong**

- **D:** More input context may help some tasks, but it does not make validation results or failures inspectable.
- **B:** Reviewing only the final diff hides tool use and validation behavior that session logs and PR artifacts can expose.
- **E:** A weekly meeting slows delivery and does not directly solve silent failures or missing structured output.

💡 For anti-pattern questions, match the remedy to the failure mode. Silent failures require explicit surfacing; unstructured work requires structured artifacts.

Q3**Domain F: Guardrails, safety, accountability, and governance**

single choice

hard

A Copilot cloud agent pull request modifies application code and also changes a deployment workflow under `.github/workflows`. The team needs CI results, but the repository uses the default setting that blocks Actions workflow runs on Copilot-created PRs. What should the reviewer do?

The workflow can access repository secrets during normal deployment jobs, and the reviewer has write access.

- A Disable validation tools temporarily so Copilot can mark the pull request ready faster.
- B Merge the agent branch into main and let protected-branch checks run afterward.
- C Inspect the workflow changes, then click Approve and run workflows for the PR. ✓**
- D Enable automatic workflow runs for future Copilot PRs to reduce reviewer interruptions.

✓ **Why the correct answer is right**

The default workflow approval gate exists because Actions workflows can be privileged and access sensitive secrets. The correct path is to inspect the proposed workflow changes and then explicitly approve the run.

✗ **Why the other options are wrong**

- **A:** Disabling validation reduces security review rather than addressing the privileged workflow execution risk.
- **B:** Merging before checks and review bypasses the intended pull request and protected-branch control path.
- **D:** Automatic workflow runs may expose repository write access or Actions secrets to unreviewed agent-authored workflow changes.

💡 For Copilot PRs that change workflows, look for an answer that keeps the approval gate and asks a human to inspect before running privileged automation.

workflow-approval

human-review

actions-guardrails

Q4**Domain A: Agent architecture and SDLC integration**

single choice

hard

A repository owner is designing the standard workflow for issues assigned to Copilot cloud agent. The team wants Copilot to make routine code changes but preserve the same contributor model used for human developers. Which workflow should the owner document?

The repository requires pull request approval before merge, and the team wants to avoid special bypasses for agent-created branches.

- A** Let the issue assigner approve the Copilot PR after CI passes, then merge through the normal button.
- B** Have Copilot push to the default branch only for small fixes, and use PRs for larger changes.
- C** Have Copilot open a draft PR, then require human readiness and an independent approval before merge. ✓
- D** Let Copilot mark the draft PR ready, request its own review, and merge after required checks pass.

✓ **Why the correct answer is right**

Copilot cloud agent works through draft pull requests and must be reviewed like any other contribution. GitHub documentation states that Copilot cannot mark its PRs ready for review, approve, or merge them, and that the person who initiated the Copilot PR cannot count as the required approving reviewer.

✗ **Why the other options are wrong**

- **A:** The initiating user's approval does not count toward required approval for a Copilot-created pull request.
- **D:** Copilot cannot mark its own pull requests ready for review, approve them, or merge them.
- **B:** Direct default-branch changes would bypass the PR-based agent workflow and reduce traceability.

💡 When a question mentions the Copilot contributor model, look for independent human review and PR-based flow rather than self-approval or direct branch updates.

contributor-model

pull-request-workflow

human-review

Q5**Domain E: Multi-agent orchestration and incident response**

single choice

medium

A repository team is retiring a custom incident-summary agent because a newer agent now creates better postmortem summaries. The team must prevent future use while preserving evidence from previous incident responses. What should the team do?

The retired agent was defined as a repository custom agent profile and has been used in several completed outage reviews.

- A** Remove the agent profile in a reviewed commit and reference the retirement reason in the commit message. ✓
- B** Keep the agent file but blank out its prompt so old sessions remain easier to recognize.
- C** Rename the old file randomly so Copilot stops matching it during future incident tasks.
- D** Leave the agent enabled and tell users to avoid selecting it in future incident reviews.

✓ **Why the correct answer is right**

Repository custom agents are profile files committed under the agents directory. Removing the profile through normal Git history prevents future selection while preserving prior commits, PRs, and session evidence for audit.

✗ **Why the other options are wrong**

- **B:** A blank prompt keeps a misleading profile present and can create confusing behavior instead of cleanly retiring the agent.
- **C:** A random rename obscures intent and weakens auditability because reviewers cannot easily understand why the agent changed.
- **D:** Relying on user convention leaves a deprecated agent available and increases the chance it is selected again.

💡 Lifecycle questions often reward normal Git-based change control: add, update, replace, or remove the agent definition with an auditable commit rather than hiding or informally discouraging use.

agent-lifecycle

retirement

auditability

Q6**Domain F: Guardrails, safety, accountability, and governance****single choice****medium**

A compliance team is approving Copilot cloud agent for repositories that include regulated application code. The engineering director wants feature work to keep moving quickly, but deployment and secrets-related changes require clear accountability. Which autonomy model should the platform team recommend?

The repositories already use pull requests, required checks, protected environments, and standard GitHub review workflows.

- A Allow Copilot to merge after successful CodeQL and secret scanning on its working branch.
- B Permit reads and draft PRs autonomously; gate deployments and secret changes with human approval. ✓**
- C Disable cloud agent outside sandbox repositories until a manual security review finishes quarterly.
- D Require approval before every file read, test run, and draft PR creation in regulated repositories.

✓ **Why the correct answer is right**

The best model right-sizes autonomy: low- and medium-risk actions can proceed through draft PR artifacts and required checks, while irreversible or compliance-sensitive actions use explicit human approval.

✗ **Why the other options are wrong**

- **D:** This adds approvals to low-risk work and slows delivery without materially reducing compliance risk.
- **A:** Copilot cloud agent cannot approve or merge pull requests, and human review remains part of the control model.
- **C:** A broad sandbox-only restriction is more restrictive than necessary when GitHub controls can gate higher-risk actions.

💡 For autonomy questions, classify the action by risk first. The strongest answer usually preserves velocity for reversible work and adds checkpoints only where risk justifies them.

autonomy-levels

human-in-the-loop

risk-classification

Q7**Domain B: MCP/tool use and environment permissions****code or config artifact****very_hard**

A platform engineer reviews a custom agent profile before enabling it for production incident triage. The agent only needs to read incident records from an internal MCP server and summarize related repository files. Which change best fixes the configuration?

Security policy allows this agent to read repository files and incident records, but it must not execute commands or update external incident data.

 `.github/agents/incident-reader.agent.md`

```
---
name: incident-reader
description: Reads incidents and summarizes related code context
tools: ["*"]
mcp-servers:
  incident-mcp:
    type: "http"
    url: "https://incidents.contoso.example/mcp"
    tools: ["*"]
    headers:
      Authorization: "Bearer ${ secrets.COPILOT_MCP_INCIDENT_TOKEN }"
---
Use incident data and repository context to prepare concise incident summaries.
```

- A** Remove mcp-servers from the profile and configure the incident server at repository level.
- B** Move the incident token to Actions secrets and keep the same Authorization header.
- C** Keep all tools, but add prompt instructions prohibiting command execution and incident updates.
- D** Limit agent tools to read, search, and incident-mcp/get_incident; allowlist only that MCP tool. ✓

✓ **Why the correct answer is right**

The custom agent tools list filters built-in and MCP tools available to the agent. The MCP server configuration also supports listing enabled tools, so both layers should expose only the repository read/search tools and the required read-only incident tool.

✗ **Why the other options are wrong**

- **C:** Prompt instructions do not remove the ability to call write-capable or execute tools.
- **B:** Copilot cloud agent uses dedicated Agents secrets and variables, not Actions secrets.
- **A:** Repository-level configuration would make the server broadly available during agent tasks rather than restricting this profile.

💡 In artifact questions, inspect both the outer agent tools list and the MCP server tools list. A wildcard at either layer may overexpose capabilities.

Q8

Domain C: Memory, state, and execution

single choice

hard

A developer uses Copilot CLI to run a refactoring task in a shared repository. The developer expects their personal formatting preference to influence the task, but does not want that preference to affect teammates' later CLI runs or code reviews. What should the team rely on?

The organization has enabled Copilot Memory. The preference is about the developer's preferred interaction and coding style, not a repository-wide convention.

- A** Keep it as a user-level preference because Copilot CLI applies preferences for the initiating user. ✓
- B** Promote the preference to a repository-level fact so all Copilot features apply it consistently.
- C** Put the preference in the repository README so Copilot code review can enforce it.
- D** Disable Copilot Memory and pass the preference manually in every CLI prompt.

✓ **Why the correct answer is right**

User-level preferences capture an individual's coding style and workflow patterns and are available only to that user. Copilot CLI applies stored facts and preferences for the user who initiated the operation.

✗ **Why the other options are wrong**

- **B:** A personal preference should not be promoted to repository-level memory because repository facts become available to users with memory access for that repository.
- **C:** A README change would turn an individual style preference into project documentation and could incorrectly influence repository behavior.
- **D:** Disabling memory removes the feature that provides the desired scoped persistence and forces repetitive prompting.

💡 When a scenario says the knowledge should follow one person but not the repo, choose user-level memory rather than repository facts or documentation changes.

Q9**Domain F: Guardrails, safety, accountability, and governance****matching magnet****very_hard**

An enterprise governance board is assigning autonomy levels for Copilot cloud agent across repositories that support a customer-facing SaaS product. Match each governance requirement to the best GitHub control pattern.

The board wants autonomy for low- and medium-risk development tasks, explicit authorization for sensitive operational actions, and traceability for agent-generated work.

Correct matches:

Allow routine code edits to move quickly, but prevent agent-authored code from merging without independent human review.

→ Let the agent create a draft pull request, then rely on human readiness, approval, and merge controls.

Support audits that need to determine which changes were agent-generated and which person initiated the work.

→ Use Copilot-authored signed commits, session-log links, and enterprise audit events for traceability.

Permit the agent to prepare a staging deployment, but require a human decision before the deployment proceeds.

→ Use environment protection with required reviewers on the staging deployment job, not on ordinary coding steps.

Choices: Require a human approval before every read, search, and local test command performed by the agent., Use environment protection with required reviewers on the staging deployment job, not on ordinary coding steps., Let the agent create a draft pull request, then rely on human readiness, approval, and merge controls., Use Copilot-authored signed commits, session-log links, and enterprise audit events for traceability., Use Copilot content exclusions as the primary control for cloud agent access to protected workflow files., Disable validation tools and depend on post-merge audit sampling to find agent-generated policy violations.

✓ Why the correct answer is right

Routine code edits fit an autonomous-with-artifact-review model: Copilot cloud agent can work on a branch and open a draft pull request, while GitHub prevents it from approving or merging its own PR and blocks the initiator from approving it. Auditability is supported by Copilot-authored signed commits, session-log links, and enterprise audit events. A staging deployment is higher risk than code exploration or editing, so an environment protection gate with required reviewers places human judgment on the deployment action rather than on every low-risk step.

💡 When classifying autonomy, choose the narrowest human checkpoint that materially reduces risk. GH-600 questions often penalize both extremes: approving every harmless action and relying only on audits after risky actions complete.

autonomy-classification

human-in-the-loop

auditability

deployment-governance

Q10**Domain C: Memory, state, and execution**

sequence order

hard

A repository recently migrated its test framework from Jest to Vitest. Copilot code review is still making suggestions that reference the old Jest workflow, and the repository owner wants to correct repository-level memory without disabling Copilot Memory for the whole organization. In which order should the owner proceed?

The team wants future Copilot cloud agent and code review activity to rely on current repository evidence while preserving useful memories that still match the codebase.

Correct order:

1. Open the repository settings and view Copilot Memory entries for the affected repository.
2. Identify memory entries whose cited evidence no longer matches the current test framework documentation.
3. Delete misleading repository-level memories instead of disabling Copilot Memory across unrelated repositories.
4. Trigger later Copilot work from the current branch so replacement facts can be grounded in current citations.

✓ **Why the correct answer is right**

Repository owners can review stored repository memories and delete entries they consider misleading or incorrect. Because Copilot validates repository facts against citations in the current branch before using them, the owner should inspect the affected repository's memories, remove obsolete entries, and allow later Copilot activity on the updated codebase to establish replacement facts.

✗ **Why the other options are wrong**

- **A:** Starting new Copilot work before removing the obsolete memory can preserve the same misleading review behavior.
- **B:** Deleting memory entries is appropriate only after identifying which stored facts are stale or misleading.
- **D:** Citation review requires first opening the repository's Copilot Memory management view.

💡 When a memory is stale, prefer targeted curation and current-branch validation. Broadly disabling memory is usually an overcorrection unless the requirement is organizational policy.

copilot-memory-curation

stale-context-prevention

memory-validation

Q11

Domain B: MCP/tool use and environment permissions

single choice

hard

A documentation agent in repo-a must read issue labels and pull request metadata from repo-b when drafting release notes. The default GitHub MCP server only returns data from repo-a. What should the administrator configure?

The agent does not need write access to repo-b, and the organization wants the smallest possible expansion of access.

- A Grant the repository GITHUB_TOKEN organization-wide read access so both repositories are visible.
- B Copy repo-b issues into repo-a so the default GitHub MCP token can read them.
- C Provide a fine-grained PAT with read-only access to repo-b through the GitHub MCP configuration. ✓
- D Enable all GitHub MCP toolsets and rely on the default token to expand repository scope.

✓ **Why the correct answer is right**

The built-in GitHub MCP server uses a specially scoped token with read-only access to the current repository by default. GitHub documents using an appropriately scoped fine-grained PAT when Copilot needs GitHub data outside the current repository.

✗ **Why the other options are wrong**

- **A:** The default repository token is not the documented mechanism for expanding GitHub MCP access across repositories.
- **B:** Duplicating issue data creates stale context and avoids the supported MCP access configuration.
- **D:** Toolsets control available GitHub MCP tools, not the repository scope of the default token.

💡 Separate tool breadth from data scope. Enabling more tools does not automatically grant access to more repositories.

github-mcp-server

repository-scope

fine-grained-pat

Q12

Domain F: Guardrails, safety, accountability, and governance

multi select

medium

A platform team is defining which Copilot cloud agent actions require human-in-the-loop approval rather than ordinary artifact review. The policy should preserve velocity for reversible development work while protecting critical operational assets. Select TWO actions that should require explicit human approval.

The team uses GitHub pull requests for code review and protected environments for deployments.

- A Running npm test in the agent's ephemeral GitHub Actions development environment.
- B Changing repository agent secrets used by deployment or incident-response tooling. ✓
- C Deploying a database migration to production using an environment-protected workflow. ✓
- D Reading repository files to summarize ownership boundaries before creating an implementation plan.
- E Updating unit tests and opening a draft pull request on a copilot branch.

✓ **Why the correct answer is right**

Production migrations and secret changes are compliance-sensitive and potentially irreversible. They should follow controlled paths with explicit authorization rather than relying only on post-hoc review.

✗ **Why the other options are wrong**

- **D:** Read-only inspection is low risk and can normally proceed without a human checkpoint.
- **E:** Draft PRs and test updates are reviewable artifacts that fit standard PR review and CI checks.
- **A:** Running tests in the agent environment is reversible and usually does not require a separate approval gate.

💡 In autonomy classification, do not equate all agent action with high risk. Reserve human-in-the-loop approval for production, secrets, billing, policy, or other sensitive changes.

critical-actions

human-in-the-loop

compliance-sensitive-changes

Q13**Domain F: Guardrails, safety, accountability, and governance**

sequence order

very_hard

A Copilot cloud agent pull request in a regulated payment-service repository modifies ``.github/copilot-instructions.md`` and ``.copilot-setup-steps.yml``, expands ``.GITHUB_TOKEN`` permissions, and asks a maintainer to approve workflow execution. Put the response steps in the correct order.

The organization requires evidence preservation, least-privilege remediation, and a path to resume the work without giving the agent broader autonomy than the task requires.

Correct order:

1. Keep the pull request draft, do not approve workflow execution, and block merge during triage.
2. Review session logs, signed commits, and audit events to identify the initiator and agent actions.
3. Replace broad setup permissions with minimum required permissions and remove unnecessary agent secrets or tools.
4. Add CODEOWNERS review for Copilot configuration files and rulesets requiring relevant checks to pass.
5. Restart the implementation under the tightened guardrails, using a new task description and normal review path.

✓ **Why the correct answer is right**

The safest sequence is to contain the immediate execution risk first by withholding workflow approval and preventing merge. Next, use the agent session logs, signed commits, and audit events to preserve accountability. Then remediate the concrete over-permissioning in setup steps and agent-accessible resources. After that, add durable governance controls such as CODEOWNERS protection and rulesets for required checks. Once the control path is fixed, the work can resume through the normal agent PR workflow.

✗ **Why the other options are wrong**

- **A:** Restarting first can reproduce the same governance gap before containment, evidence review, or permission remediation is complete.
- **B:** This is the correct first step because unreviewed agent workflow changes can access privileged Actions behavior if approved too early.
- **C:** This belongs after containment; evidence review is essential, but it should not leave workflow execution available during triage.
- **D:** This is a durable prevention step, but it should follow immediate remediation of the active over-permissioned change.
- **E:** This should occur after audit review identifies what changed, and before adding longer-term ruleset and ownership controls.

💡 Sequence questions in Domain F often reward containment, evidence, remediation, prevention, then resumption. Options that resume work before closing the guardrail gap are usually premature.

incident-containment

auditability

least-privilege

rulesets

codeowners

Q14

Domain E: Multi-agent orchestration and incident response

single choice

medium

An incident-response team wants one Copilot SDK session to triage a production regression, delegate log analysis to a read-only investigator, delegate a minimal patch to an editor, and then summarize the outcome for the incident commander. Which orchestration pattern best fits this workflow?

The team wants a single coordinating agent to keep the overall plan coherent while specialized agents execute bounded subtasks with different tool access.

- A Use a pipeline pattern so each agent transforms the previous agent's full output sequentially.
- B Use an orchestrator-worker pattern so one parent coordinates specialized sub-agents and integrates results. ✓**
- C Use independent fan-out so each agent works without a shared coordinating session.
- D Use a debate pattern so all agents propose competing patches before any triage occurs.

✓ **Why the correct answer is right**

The orchestrator-worker pattern fits because one parent session decomposes the incident task, delegates to specialized sub-agents, receives lifecycle events, and integrates their outputs into a final response.

✗ **Why the other options are wrong**

- **A:** A pipeline is better when each agent performs a single sequential transformation, not when one coordinator must route work by expertise.
- **D:** Debate is useful for comparing alternatives, but it delays triage and does not provide bounded worker execution.
- **C:** Uncoordinated fan-out risks duplicated work and inconsistent outputs because no parent agent integrates the results.

💡 For GH-600 orchestration items, identify whether the scenario needs coordination, sequential transformation, parallel independence, or independent comparison. A named coordinating parent usually signals orchestrator-worker.

orchestrator-worker

multi-agent-delegation

incident-response

Q15**Domain E: Multi-agent orchestration and incident response**

multi select

medium

A team wants to add a new performance-investigator sub-agent to an existing Copilot SDK incident workflow. Existing investigator and editor agents are already used by live sessions. Select TWO actions that support safe addition of the new agent.

The team wants future sessions to route performance incidents more accurately without changing the behavior of current in-flight sessions.

- A** Add a new customAgents entry with a distinct name, specific description, scoped tools, and prompt. ✓
- B** Remove descriptions from older agents so the new performance agent receives more tasks.
- C** Update observability handling so selected, started, completed, and failed events show the new agent. ✓
- D** Give the new agent tools:null so it can handle any unexpected incident diagnostic task.
- E** Reuse the existing investigator name so the runtime automatically replaces it mid-session.

✓ **Why the correct answer is right**

A distinct custom agent definition with a clear description helps the runtime match user intent and keeps capabilities scoped. Updating lifecycle-event handling ensures the new agent is visible in operational dashboards and post-hoc reviews.

✗ **Why the other options are wrong**

- **E:** Reusing a name risks ambiguity and does not provide a controlled replacement strategy for live work.
- **D:** tools:null grants all session tools, which weakens isolation for a diagnostic-only agent.
- **B:** Removing useful descriptions from existing agents makes routing less predictable across the workflow.

💡 When adding an agent, think in two tracks: routing quality and operational visibility. Avoid changes that blur identities or broaden tools just to make adoption easier.

add-agent

custom-agent-routing

workflow-continuity

Q16

Domain B: MCP/tool use and environment permissions

code or config artifact

medium

A team adds a setup workflow so Copilot cloud agent can install a private CLI before starting. Copilot still starts without running the setup steps. What is the primary problem in the artifact?

The file is committed to the default branch at `.github/workflows/copilot-setup-steps.yml`.

 `.github/workflows/copilot-setup-steps.yml`

```
name: Copilot Setup Steps
on:
  workflow_dispatch:
  push:
    paths:
      - .github/workflows/copilot-setup-steps.yml
jobs:
  prepare-agent:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v6
      - name: Install internal CLI
        run: npm install -g @contoso/internal-cli
```

- A The workflow must include a `pull_request` trigger before Copilot can run setup steps.
- B The checkout action must set `fetch-depth` to zero for setup steps to run.
- C The job must be named `copilot-setup-steps` for Copilot to pick it up. ✓
- D The workflow must grant `contents: write` so Copilot can modify repository files.

✓ **Why the correct answer is right**

The setup workflow must contain a single job named `copilot-setup-steps`. If the job uses another name, Copilot cloud agent will not pick it up as its setup job.

✗ **Why the other options are wrong**

- **A:** The `pull_request` trigger is useful for validation but is not the reason Copilot ignores this setup job.
- **B:** GitHub overrides checkout `fetch-depth` as needed; this is not required for setup execution.
- **D:** Setup steps should use the lowest required permissions, and Copilot receives its own token for operations.

💡 In setup-step YAML questions, check the filename, default-branch placement, and exact job name before examining dependency commands.

Q17

Domain C: Memory, state, and execution

multi select

hard

A long-running agent task is paused after implementing half of a migration. The team needs another agent session to resume without repeating completed work or drifting from the original acceptance criteria. Select TWO actions the team should include in its state strategy.

The repository uses pull requests as the system of record. The new session will start after several commits have been pushed by humans.

- A Preserve the full tool-call history in context so the agent can replay prior commands exactly.
- B Rely on the previous chat transcript because it contains the most complete record of reasoning.
- C Ask the new session to infer progress from file timestamps and recent commit messages.
- D Store a structured checkpoint in the PR description listing completed steps, open decisions, and remaining work. ✓
- E Require the agent to compare its current plan with the original issue acceptance criteria before acting. ✓

✓ **Why the correct answer is right**

A structured PR checkpoint is a durable artifact that survives session boundaries and helps resume without repeating work. Comparing the current plan to the original issue and acceptance criteria detects context drift and re-anchors the agent before additional changes.

✗ **Why the other options are wrong**

- **B:** A chat transcript may be inaccessible, too verbose, or not treated as the durable repository artifact for later sessions.
- **C:** Timestamps and commit messages are weak proxies for decisions and can cause the agent to redo or skip important steps.
- **A:** Keeping full tool-call history increases context noise; completed-step history should be pruned into a concise durable checkpoint.

💡 For resume-and-drift questions, choose durable checkpoints plus re-anchoring to the original requirement; avoid relying on raw transcripts or replaying every prior tool call.

Q18**Domain E: Multi-agent orchestration and incident response**

code or config artifact

hard

A team is building an incident console that shows which Copilot SDK sub-agent handled each part of a response. During a simulated outage, two sub-agents run in one parent session, but the console sometimes marks the wrong agent as completed. What change should the team make?

The console must reconstruct a reliable execution tree for post-hoc incident review.

 Current event handler

```
const activeAgents = new Map<string, { name: string; status: string }>();

session.on((event) => {
  if (event.type === "subagent.started") {
    activeAgents.set(event.data.agentName, {
      name: event.data.agentName,
      status: "running"
    });
  }

  if (event.type === "subagent.completed") {
    const node = activeAgents.get(event.data.agentName);
    if (node) node.status = "completed";
  }

  if (event.type === "subagent.failed") {
    const node = activeAgents.get(event.data.agentName);
    if (node) node.status = "failed";
  }
});
```

- A** Store only subagent.selected events because they contain the tool list selected by the runtime.
- B** Key execution nodes by toolCallId and update status from completed or failed events. ✓
- C** Create one parent session per sub-agent so event correlation is unnecessary during incidents.
- D** Key the map by agent display name and overwrite older entries when the same agent repeats.

✓ **Why the correct answer is right**

Sub-agent started, completed, and failed events include toolCallId values that can be used to correlate lifecycle events and reconstruct the execution tree accurately.

✗ **Why the other options are wrong**

- **D:** Display names are human-readable labels and can repeat, so they are weak identifiers for execution correlation.
- **A:** The selected event is useful for tool visibility, but it does not by itself represent execution completion or failure.
- **C:** Splitting sessions removes useful parent orchestration context and avoids the event-correlation problem instead of solving it.

💡 For GH-600 observability questions, look for stable correlation identifiers. In Copilot SDK sub-agent traces, toolCallId is the key signal for reconstructing execution.

[subagent-events](#)

[observability-ui](#)

[execution-tree](#)

Q19

Domain B: MCP/tool use and environment permissions

multi select

hard

A team wants to reuse an MCP configuration that currently works in VS Code for Copilot cloud agent. The VS Code configuration uses inputs and an envFile and exposes all server capabilities. Select THREE changes required or strongly recommended for the cloud agent configuration.

The server gives access to work items and repository metadata, and the agent will run autonomously on assigned issues.

- A Enable all server tools temporarily so the agent can discover the safest subset itself.
- B Keep the envFile setting because Copilot cloud agent expands it before starting MCP servers.
- C Replace inputs and envFile references with direct env substitutions in the MCP configuration. ✓
- D Move secret values into the repository's Dependabot secrets for shared automation use.
- E Store MCP credentials as Agents secrets or variables with names beginning COPILOT_MCP_. ✓
- F Add a tools key for each MCP server and list only the needed tools. ✓

✓ **Why the correct answer is right**

Copilot cloud agent MCP configuration requires or expects a tools key, should use direct env substitutions instead of VS Code inputs and envFile, and can reference only MCP-specific Copilot Agents secrets or variables that use the COPILOT_MCP_ prefix.

✗ **Why the other options are wrong**

- **B:** GitHub documentation says to switch envFile usage to env directly when adapting VS Code MCP settings.
- **D:** Copilot cloud agent does not read Dependabot secrets; it uses dedicated Agents secrets and variables.
- **A:** Autonomous MCP tool use makes broad temporary access risky; GitHub recommends allowlisting specific read-only tools.

💡 Migration questions often hide two issues: syntax adaptation and permission reduction. Handle both rather than only making the JSON parse.

mcp-migration

mcp-configuration

secret-substitution

Q20

Domain A: Agent architecture and SDLC integration

single choice

medium

A platform team is introducing Copilot cloud agent for small feature requests. Developers currently assign vague issues to the agent, and reviewers receive pull requests with code changes but no clear statement of intent or validation steps. What should the team change first?

The team wants faster reviews without adding a separate ticketing system or blocking every task on an architecture meeting.

- A Allow the agent to open PRs normally, then ask reviewers to infer intent from the diff.
- B Require the agent to wait for a senior engineer to write implementation details in every issue.
- C Require each agent task to produce a structured plan, expected output, and validation checklist in GitHub. ✓
- D Move agent planning notes to a shared document so pull request discussions stay concise.

✓ **Why the correct answer is right**

Domain A focuses on integrating agents into the SDLC with clear inputs, outputs, and success criteria. A structured plan and validation checklist make the agent's reasoning inspectable inside GitHub, which supports review without creating a separate control plane.

✗ **Why the other options are wrong**

- **B:** Requiring a senior engineer to pre-write every implementation detail slows delivery and does not teach the agent to produce inspectable artifacts.
- **A:** Inferring intent from the diff is a common no-structured-output anti-pattern that increases review ambiguity.
- **D:** Moving planning notes outside GitHub weakens GitHub's role as the system of record for agent work.

💡 For Domain A questions, prefer answers that make agent work visible in issues, PRs, comments, or checks rather than relying on informal context.

sdlc-integration

plan-act-evaluate

inspectable-artifacts

Q21**Domain A: Agent architecture and SDLC integration**

code or config artifact

hard

A repository frequently uses Copilot cloud agent for dependency update tasks. The agent wastes time rediscovering the build environment, even though the team added a setup workflow. What is the minimal fix to make the setup workflow part of the agent SDLC path?

The team wants Copilot's environment prepared before the agent starts work, without changing runner networking or adding external tools.

 `.github/workflows/copilot-setup-steps.yml`

```
name: Copilot Setup Steps

on:
  workflow_dispatch:
  push:
    paths:
      - .github/workflows/copilot-setup-steps.yml
  pull_request:
    paths:
      - .github/workflows/copilot-setup-steps.yml

jobs:
  setup:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Checkout code
        uses: actions/checkout@v5
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"
      - name: Install dependencies
        run: npm ci
```

- A Rename the workflow file to setup.yml and reference it from each agent prompt.
- B Rename the job to copilot-setup-steps and keep the workflow on the default branch. ✓**
- C Move dependency installation into the issue template so Copilot repeats it per task.
- D Add a matrix strategy so Copilot can choose the best runtime during execution.

✓ **Why the correct answer is right**

The Copilot setup workflow must be located at `.github/workflows/copilot-setup-steps.yml` and contain a single job named `copilot-setup-steps`. The workflow also must be present on the default branch before Copilot will use it.

✗ Why the other options are wrong

- **A:** Changing the filename prevents Copilot from recognizing the special setup workflow.
- **D:** A matrix strategy does not address the required job name and is outside the supported minimal fix.
- **C:** Putting setup commands in issue text is slower and less reliable than preconfiguring the agent environment.

💡 For copilot-setup-steps questions, verify the exact file path, job name, default-branch requirement, and supported job settings before considering broader workflow changes.

copilot-setup-steps

agent-runtime-preparation

sdlc-reliability

Q22**Domain D: Evaluation, telemetry, error analysis, and tuning**

multi select

medium

A security engineering team evaluates Copilot-created dependency update pull requests. The team wants automated evaluation signals that indicate whether an agent improved or worsened supply-chain risk. Select TWO signals the team should prioritize.

The repository already uses GitHub code scanning, dependency review, and secret scanning in pull request workflows.

- A** New dependency review findings for high or critical vulnerabilities introduced by the pull request. ✓
- B** Total number of Copilot comments in the pull request timeline before reviewer assignment.
- C** Whether the pull request title follows the team's preferred conventional commit format.
- D** Number of files modified by the agent compared with the average human dependency pull request.
- E** New code scanning alerts or unresolved CodeQL findings associated with the agent's changes. ✓

✓ **Why the correct answer is right**

Dependency review findings and code scanning alerts are automated, quantitative signals that directly measure whether the change introduced security risk or unresolved analysis findings.

✗ **Why the other options are wrong**

- **D:** File count may help review sizing, but it does not measure supply-chain vulnerability impact.
- **B:** Comment volume is activity telemetry and does not indicate whether the dependency update is safer.
- **C:** Title format is useful hygiene, but it is not a security evaluation signal.

💡 When the stem asks for automated evaluation signals, choose scan outputs tied to the risk being evaluated rather than workflow cosmetics.

automated-scanning

quantitative-signals

security-evaluation

Q23

Domain B: MCP/tool use and environment permissions

code or config artifact

very_hard

A team pastes the following MCP configuration from a developer's VS Code workspace into the Copilot cloud agent MCP settings. Validation fails, and the security review also flags excessive tool exposure. Which revision best addresses both issues?

Copilot needs only to read service tickets from the service desk MCP server during assigned tasks.

 MCP configuration pasted into repository settings

```
{
  "mcpServers": {
    "servicedesk": {
      "command": "npx",
      "args": ["-y", "@contoso/servicedesk-mcp", "--token", "${input:ticketToken}"],
      "inputs": [
        {
          "id": "ticketToken",
          "type": "promptString",
          "password": true
        }
      ],
      "envFile": ".env.local"
    }
  }
}
```

- A Add tools: ["*"] and keep inputs so Copilot can prompt for the token.
- B Add a specific tools list and replace inputs and envFile with env using a COPILOT_MCP_ secret. ✓**
- C Remove envFile only; VS Code inputs are supported in repository MCP settings.
- D Move this configuration to copilot-setup-steps.yml so Actions can resolve inputs.

✓ **Why the correct answer is right**

When adapting VS Code MCP settings for Copilot cloud agent, GitHub directs teams to add a tools key, switch inputs to env directly, switch envFile to env directly, and update argument references to use environment variables. MCP secrets must be supplied as Agents secrets or variables with the COPILOT_MCP_ prefix.

✗ **Why the other options are wrong**

- **A:** Using tools: ["*"] grants broad autonomous access, and VS Code inputs are not the cloud agent pattern.
- **C:** Inputs also need to be replaced, and this option does not restrict MCP tools.

- **D:** copilot-setup-steps.yml prepares the environment; it is not where repository MCP JSON is configured.

💡 For config-artifact items, solve parse/runtime compatibility first, then verify least privilege. The best answer usually fixes both.

mcp-json

vs-code-migration

mcp-tool-allowlist

Q24

Domain B: MCP/tool use and environment permissions

multi select

hard

An external audit MCP server is configured for Copilot cloud agent, but the server receives an empty Authorization header. The repository has a GitHub Actions secret named AUDIT_TOKEN. Select TWO changes that align with GitHub's supported secret handling for MCP configuration.

The MCP server is remote HTTP, and its documentation requires Authorization: Bearer <token>.

- A Move the token into Codespaces secrets so development tools and agents can share it.
- B Create an Agents secret named COPILOT_MCP_AUDIT_TOKEN with the audit token value. ✓
- C Reference the token in the MCP headers using \$COPILOT_MCP_AUDIT_TOKEN or \${COPILOT_MCP_AUDIT_TOKEN}. ✓
- D Place the token directly in the MCP JSON so the header is not empty.
- E Rename the existing Actions secret to COPILOT_MCP_AUDIT_TOKEN and keep it under Actions.

✓ **Why the correct answer is right**

MCP configuration can use header substitutions that reference Copilot Agents secrets or variables. For MCP servers, the referenced secret or variable name must begin with COPILOT_MCP_, and supported syntax includes \$VAR and \${VAR}.

✗ **Why the other options are wrong**

- **E:** Renaming an Actions secret is insufficient because Copilot cloud agent does not read Actions secrets.
- **D:** Embedding the token directly in MCP JSON exposes a credential unnecessarily and bypasses supported secret handling.
- **A:** Copilot cloud agent does not read Codespaces secrets for MCP configuration.

💡 Secret questions often use a correctly named secret in the wrong store. Verify both the prefix and the secret type.

mcp-secrets

headers

secret-prefix

Q25

Domain B: MCP/tool use and environment permissions

single choice

medium

Copilot cloud agent opens a pull request but reports that npm install failed because requests to `https://packages.contoso.example/npm/internal` were blocked by the agent firewall. The repository uses a private package registry only for this project. What should the repository administrator do?

The organization allows repository administrators to add custom firewall rules, and security wants internet access to remain restricted.

- A Disable the Copilot cloud agent firewall for the repository until dependency installation succeeds.
- B Store the registry hostname in an Agents variable and rerun the same task.
- C Add a custom allowlist rule for the required registry URL or domain only. ✓**
- D Move npm install into a normal CI workflow after the agent creates the pull request.

✓ **Why the correct answer is right**

The Copilot cloud agent firewall is designed to restrict outbound access and supports custom allowlist rules for domains or URLs. Adding only the required registry path or domain preserves the security boundary while enabling dependency installation.

✗ **Why the other options are wrong**

- **A:** Disabling the firewall increases data-exfiltration risk and is broader than needed.
- **B:** An environment variable does not permit network traffic blocked by the firewall.
- **D:** Deferring installation to CI prevents the agent from validating changes in its own environment.

💡 For firewall questions, prefer narrowly allowing the required destination over disabling controls or moving validation out of the agent path.

firewall

network-allowlist

environment-constraints

Q26**Domain E: Multi-agent orchestration and incident response**

code or config artifact

hard

A repository owner reviews a custom Copilot cloud agent profile for incident postmortems. The agent should be available only when a human intentionally selects it after an incident, and it should not edit files. Which revision best meets the requirement?

The current profile is too broad and sometimes appears for general troubleshooting prompts.

 .github/agents/postmortem.agent.md

```
---
name: Postmortem Writer
description: Writes incident postmortems, analyzes logs, edits follow-up issues,
and updates runbooks
target: github-copilot
tools: ["*"]
disable-model-invocation: false
---
You write incident postmortems and update repository documentation after outages.
```

- A** Set `disable-model-invocation:true` and restrict tools to read, search, and approved incident MCP read tools. ✓
- B** Keep tools wildcard and shorten the description to 'Handles incident work' for broader reuse.
- C** Set `user-invocable:false` and keep edit access so only automation can run postmortem updates.
- D** Remove target and tools so the profile inherits defaults from every supported environment.

✓ **Why the correct answer is right**

`disable-model-invocation:true` prevents Copilot cloud agent from automatically using the custom agent based on task context, requiring manual selection. Restricting tools to read/search and read-only incident tools preserves the postmortem role without file-edit capability.

✗ **Why the other options are wrong**

- **B:** A broad description and wildcard tools increase the chance of over-selection and over-permissioning.
- **C:** `user-invocable:false` prevents manual selection, which conflicts with the requirement for intentional human use.
- **D:** Inheriting default tools can grant broader access than intended and makes behavior less predictable across environments.

💡 For cloud-agent profile questions, distinguish manual invocation from automatic model invocation. Also check whether omitted or wildcard tools accidentally grant broader access.

agent-profile-configuration

manual-selection

least-privilege-orchestration

Q27**Domain D: Evaluation, telemetry, error analysis, and tuning**

single choice

hard

A team uses a custom implementation-planner agent before assigning implementation work to Copilot cloud agent. A recent plan proposed adding a server-side validation layer, but the final pull request only added client-side checks and updated screenshots. What is the best first analysis step?

The repository requires a durable plan artifact for each feature, and the team wants to improve plan adherence before changing agent permissions.

- A** Compare the structured plan sections, commit history, and final diff to locate the divergence point. ✓
- B** Replace the implementation agent with a documentation agent that records more detailed assumptions.
- C** Compare median time to merge for this pull request against other Copilot-created pull requests.
- D** Ask reviewers to approve the pull request, then collect feedback during the next retrospective.

✓ **Why the correct answer is right**

The best first step is error analysis using the plan, commit history, and final diff. This identifies whether the agent misunderstood the task, lost context, or deviated during execution.

✗ **Why the other options are wrong**

- **C:** Time to merge is a throughput metric and does not identify why the implementation diverged from the plan.
- **D:** Merging before analysis accepts an unverified result and delays root-cause discovery.
- **B:** Changing agents adds complexity before the team understands the failure mode.

💡 For plan-adherence failures, inspect the plan-to-commit-to-diff chain before tuning tools, models, or workflow structure.

plans-vs-diffs

reasoning-errors

implementation-adherence

Q28

Domain B: MCP/tool use and environment permissions

matching magnet

hard

A platform team is hardening several Copilot cloud agent integrations after a pilot revealed inconsistent MCP behavior across repositories. Match each observed situation to the best configuration decision.

The organization uses GitHub Copilot Business. Repository administrators can configure cloud agent MCP settings, while organization owners manage MCP registry policy and shared Agents secrets.

Correct matches:

An incident MCP server requires an API token, and the repository MCP JSON currently references \$INCIDENT_API_TOKEN in a header. → **Create an Agents secret prefixed with COPILOT_MCP_ and reference that value from the MCP headers or env.**

Developers can discover useful MCP servers, but security wants Copilot use limited to a curated company catalog. → **Configure an MCP registry URL and set MCP access restriction to registry servers for the organization.**

A custom triage agent needs current-repository GitHub issue and pull request context, but no external service tools. → **Limit the custom agent tools to the needed GitHub MCP tool names or aliases instead of enabling all tools.**

A team copied a remote MCP configuration that relies on an OAuth browser authorization flow into Copilot cloud agent. → **Replace the remote OAuth dependency with a supported PAT or non-OAuth authentication pattern for cloud agent.**

Choices: Create an Agents secret prefixed with COPILOT_MCP_ and reference that value from the MCP headers or env., Configure an MCP registry URL and set MCP access restriction to registry servers for the organization., Limit the custom agent tools to the needed GitHub MCP tool names or aliases instead of enabling all tools., Replace the remote OAuth dependency with a supported PAT or non-OAuth authentication pattern for cloud agent., Store the token as an Actions secret and reference it directly from the MCP configuration., Enable all configured MCP tools so Copilot can choose the safest tool dynamically.

✓ **Why the correct answer is right**

Copilot cloud agent MCP configuration only receives secret or variable substitutions intended for MCP when the Agents secret or variable name starts with COPILOT_MCP_. Organization and enterprise owners can curate MCP discovery and usage by configuring a registry URL and restricting access to registry servers. Custom agents should expose only the specific tools needed for the task. Copilot cloud agent documentation notes that remote MCP servers using OAuth are not currently supported for cloud agent, so a supported non-OAuth pattern is required.

💡 For GH-600 MCP questions, separate three control planes: repository MCP JSON, organization MCP registry policy, and Agents secrets. Many distractors mix these with Actions secrets or broad tool enablement.

mcp-configuration

agent-secrets

mcp-allowlist

least-privilege

Q29

Domain A: Agent architecture and SDLC integration

code or config artifact

hard

A platform engineer is defining a custom agent used only when a human intentionally selects it for implementation planning. The current profile uses an older field and may be invoked automatically for tasks that merely mention planning. Which change is the best current configuration?

The agent should remain selectable by users, but Copilot cloud agent should not automatically choose it based on task context.

 .github/agents/release-plan.agent.md

```
---
name: release-plan
description: Creates release implementation plans and acceptance criteria
infer: false
tools: ["read", "search", "edit"]
---
```

Create a structured release plan in Markdown. Include assumptions, dependencies, acceptance criteria, and validation steps. Do not implement release code.

- A Remove tools because an agent without tools is less likely to make code changes.
- B Keep infer: false because it is the recommended current syntax for manual-only selection.
- C Replace infer: false with disable-model-invocation: true and keep the agent user-invocable. ✓**
- D Replace infer: false with user-invocable: false so users cannot select the agent accidentally.

✓ Why the correct answer is right

GitHub's current custom agent configuration uses disable-model-invocation: true to prevent Copilot cloud agent from automatically using the custom agent based on context. user-invocable remains true by default, so humans can still select it.

✗ Why the other options are wrong

- **D:** user-invocable: false would prevent manual user selection, which conflicts with the scenario.
- **A:** Removing tools would prevent the agent from creating a useful planning artifact in the repository.
- **B:** infer is retired; current guidance says to use disable-model-invocation and user-invocable instead.

💡 Expect GH-600 beta items to prefer current field names. Treat retired compatibility fields as distractors when a current syntax is available.

custom-agent-configuration

manual-agent-selection

agent-lifecycle

Q30**Domain D: Evaluation, telemetry, error analysis, and tuning****code or config artifact****very_hard**

A platform team builds a Copilot SDK service that delegates repository triage tasks to agents. Incident reviews show request logs in the web service, but no distributed trace connects the incoming request to the Copilot session and tool handler. Which change best closes the telemetry gap?

The team already uses OpenTelemetry in the web service and wants tool invocations to appear under the same trace as the request that started the agent task.

 **agent-service.ts**

```
import { CopilotClient } from "@github/copilot-sdk";

export function createClient() {
  return new CopilotClient({});
}

export async function runTriage(prompt: string) {
  const client = createClient();
  const session = await client.createSession();
  await session.send(prompt);
}
```

- A** Configure telemetry on CopilotClient and propagate W3C trace context into session RPC calls. ✓
- B** Replace distributed tracing with Copilot usage metrics grouped by organization and model name.
- C** Add console logging around session creation and correlate requests by timestamp during incident reviews.
- D** Store the prompt and response in a database table keyed by repository and pull request number.

✓ **Why the correct answer is right**

The SDK supports telemetry configuration and W3C Trace Context propagation. Providing telemetry and trace context links SDK, CLI, and tool-handler spans into a distributed trace.

✗ **Why the other options are wrong**

- **C:** Timestamp correlation is fragile and does not create a trace across SDK and tool boundaries.
- **D:** Persisting prompts helps auditability but does not connect runtime spans for failure analysis.
- **B:** Usage metrics are useful for adoption and throughput analysis, but they do not replace per-request traces.

💡 For telemetry-gap questions, distinguish aggregate metrics from traceability. Distributed failures

require trace context, not only logs or dashboards.

opentelemetry

trace-context

telemetry-gap

Q31**Domain E: Multi-agent orchestration and incident response****code or config artifact****very_hard**

A team creates an incident rollback sub-agent in the Copilot SDK. During testing, the runtime automatically selects the rollback agent for vague prompts such as 'clean up after the outage.' The team wants the parent agent to coordinate incidents while rollback execution occurs only after explicit selection. Which change best satisfies the requirement?

The rollback tool is registered at the session level because it must be available to the rollback sub-agent, but the parent coordinator should not call it directly.

 Current Copilot SDK session configuration

```
const session = await client.createSession({
  model: "gpt-4.1",
  tools: [rollbackDeploymentTool, createAuditNoteTool, inspectLogsTool],
  customAgents: [
    {
      name: "rollback",
      displayName: "Rollback Agent",
      description: "Rolls back deployments and removes temporary incident changes",
      tools: null,
      prompt: "Execute approved rollback steps and record evidence."
    },
    {
      name: "investigator",
      displayName: "Investigator Agent",
      description: "Reads logs and summarizes likely root cause",
      tools: ["inspect-logs", "create-audit-note"],
      prompt: "Investigate without changing production state."
    }
  ]
});
```

- A Remove the rollback agent description so intent matching has less information available.
- B Set infer:false on rollback, scope its tools explicitly, and exclude rollback from the default agent. ✓**
- C Move rollbackDeploymentTool into the parent tools array and ask the parent to avoid it.
- D Keep tools:null on rollback so it can choose whatever incident tool seems useful.

 Why the correct answer is right

Setting infer:false prevents automatic selection for vague prompts. Explicit tool scoping applies least privilege to the rollback sub-agent, and defaultAgent.excludedTools hides the rollback tool from the main coordinator while keeping it available to the specialized sub-agent.

 Why the other options are wrong

- **A:** Weakening the description makes routing less predictable and does not prevent automatic invocation reliably.
- **C:** Putting the rollback tool directly in the parent agent's visible tool set undermines the desired coordinator-executor separation.
- **D:** Using tools:null gives the rollback agent all session tools, which increases blast radius during incident response.

💡 For custom-agent orchestration items, combine selection control with tool isolation. The best answer usually changes both when an agent can be invoked and what it can do.

custom-agent-isolation

explicit-delegation

agent-exclusive-tools

Q32**Domain D: Evaluation, telemetry, error analysis, and tuning**

single choice

hard

A mobile repository's Copilot agent repeatedly reformats generated localization files while fixing small UI bugs. Reviewers spend most of their time reverting unrelated diffs. Which tuning change best addresses the observed failure without blocking useful agent work?

Session logs show the agent used broad shell commands to run a formatter across the repository after editing one component.

- A Disable Copilot cloud agent for the repository until localization file ownership is redesigned.
- B Add instructions limiting edits to task files and require targeted validation before creating the pull request. ✓**
- C Ask reviewers to leave more comments so the agent can repair localization files afterward.
- D Measure total changed lines per pull request and reject sessions above the monthly average.

✓ **Why the correct answer is right**

The observed failure is overly broad tool behavior and missing operational constraints. Targeted instructions and validation tune behavior while preserving the agent's ability to make useful scoped changes.

✗ **Why the other options are wrong**

- **A:** Disabling the agent removes productive work instead of correcting the specific failure pattern.
- **D:** Changed-line thresholds are a coarse evaluation signal and may reject legitimate refactors without preventing the root cause.
- **C:** Relying on reviewer comments shifts the cost to humans after the avoidable diffs are already created.

💡 Tuning questions reward narrow, evidence-based adjustments to instructions, constraints, memory, or tools instead of broad shutdowns.

tuning-instructions

tool-access-refinement

error-analysis

Q33

Domain F: Guardrails, safety, accountability, and governance

multi select

very_hard

A healthcare software group is writing its minimum control baseline for Copilot cloud agent. The baseline must reduce vulnerable code risk, prevent bypass of required approvals, and preserve traceability for audits. Select THREE controls that satisfy the baseline.

The group wants to use GitHub's built-in controls where they are sufficient instead of building a separate approval portal.

- A Allow the triggering developer to satisfy required approval after checking session logs.
- B Keep Copilot's security validation tools enabled for agent-generated code before PR completion. ✓**
- C Store production tokens as Actions secrets and reference them from agent instructions.
- D Disable default workflow approval so CI findings appear before reviewers inspect changes.
- E Preserve human review and merge for draft PRs created by Copilot cloud agent. ✓**
- F Use session logs and audit events to trace agent actions during reviews. ✓**

✓ **Why the correct answer is right**

Security validation helps catch vulnerabilities, hardcoded secrets, and insecure dependencies. Human review and merge preserve approval controls. Session logs, signed commits, and audit events support traceability.

✗ **Why the other options are wrong**

- **D:** Removing workflow approval increases exposure to privileged workflow execution before human inspection.
- **A:** GitHub prevents the user who triggered the agent from approving its PR, preserving separation of duties.
- **C:** Agents should not access Actions secrets; Copilot cloud agent has a separate secrets and variables mechanism.

💡 Multi-select guardrail questions often mix prevention, review, and auditability. Choose controls that cover all required risk categories rather than several controls from one category.

built-in-protections

auditability

review-controls

Q34

Domain A: Agent architecture and SDLC integration

single choice

very_hard

A team is enabling Copilot cloud agent in a repository with existing Actions workflows. They want the agent to create draft PRs quickly, but they do not want workflows from agent pushes to run before a human has looked at the proposed work. Which workflow design best fits this requirement?

The team is optimizing the agent SDLC path, not changing network restrictions, secrets access, or compliance policy.

- A** Use the default approval flow: review the draft PR, then approve and run workflows from the PR. ✓
- B** Disable Actions on Copilot branches and ask the agent to summarize expected test results.
- C** Require Copilot to merge only after workflows pass and skip the draft PR stage.
- D** Enable automatic workflow runs for every Copilot push so CI feedback arrives before human review.

✓ **Why the correct answer is right**

By default, GitHub Actions workflows do not run automatically when Copilot pushes changes to a pull request. A human can review the draft PR and then use the Approve and run workflows control, preserving a lightweight checkpoint without blocking PR creation.

✗ **Why the other options are wrong**

- **D:** Automatic workflow runs prioritize speed but remove the requested human checkpoint before workflows run.
- **B:** Disabling Actions entirely loses automated validation and relies on unverifiable summaries.
- **C:** Copilot cannot merge its own pull requests, and skipping drafts conflicts with the normal agent workflow.

💡 For autonomy-level questions in Domain A, choose the option that places the checkpoint at the specific risky transition rather than disabling the workflow or removing automation.

agent-workflow-control

draft-pr-process

actions-approval

Q35**Domain E: Multi-agent orchestration and incident response**

multi select

hard

A platform team runs three Copilot-assisted remediation agents in parallel after an outage. Two agents independently edit the same retry middleware, while the third updates documentation. The orchestrator detects incompatible code changes before merge. Select TWO actions the team should take.

The organization wants to preserve useful work while preventing contradictory changes from entering the default branch.

- A** Merge the first completed remediation PR and close the second without reviewing its session evidence.
- B** Discard all three branches because one conflict makes the overall incident workflow unreliable.
- C** Allow the documentation PR to continue if it does not depend on the disputed code decision. ✓
- D** Pause the conflicting code PRs and have a human compare the branches and session artifacts. ✓
- E** Ask both agents to continue editing the same middleware until one branch passes CI.

✓ **Why the correct answer is right**

Pausing the conflicting code PRs preserves the competing work while requiring human resolution of contradictory outputs. Letting the independent documentation PR continue avoids unnecessary cancellation of unaffected work.

✗ **Why the other options are wrong**

- **A:** Closing one PR without reviewing evidence can discard the better fix and weakens post-incident auditability.
- **E:** Continuing simultaneous edits to the disputed middleware can deepen the conflict instead of resolving the dependency.
- **B:** Discarding all branches loses unrelated valid work and treats a localized conflict as a total workflow failure.

💡 In multi-agent conflict questions, avoid extremes: do not merge blindly, and do not cancel unaffected work. Preserve evidence, isolate the conflict, and resolve the disputed decision explicitly.

parallel-execution

agent-conflict-resolution

operational-signals

Q36**Domain A: Agent architecture and SDLC integration****code or config artifact****very_hard**

A team creates a custom agent to produce implementation plans before a coding agent starts work. Reviewers find that the agent sometimes edits application code while planning. Which change best preserves planning-versus-execution separation while still producing an inspectable plan?

The team wants the planning agent to create a Markdown plan in the repository and leave implementation to a separate coding agent after review.

 `.github/agents/implementation-planner.agent.md`

```
---
name: implementation-planner
description: Creates implementation plans for medium-risk feature work
tools: ["read", "search", "edit", "execute"]
---
```

You analyze the issue, decide on an implementation approach, update files as needed, and summarize the changes in a pull request.

- A** Limit tools to read, search, and edit, and prompt the agent to create only a plan document. ✓
- B** Remove the description so Copilot infers the agent's role from the filename and prompt.
- C** Add `disable-model-invocation: true` so the agent cannot reason about implementation details.
- D** Set tools to `["*"]` so the agent can validate the plan using every available tool.

✓ **Why the correct answer is right**

A planning agent still needs to inspect the repository and write a planning artifact, but it should not execute commands or implement code. GitHub custom agent configuration supports specifying a subset of tools, and the official implementation-planner example uses read, search, and edit to create structured Markdown plans.

✗ **Why the other options are wrong**

- **B:** The description is required for a custom agent profile and helps define the agent's purpose.
- **D:** Enabling all tools expands the agent's action surface and weakens planning-versus-execution boundaries.
- **C:** `disable-model-invocation` controls automatic selection behavior, not whether the agent can reason once selected.

💡 For planning-agent questions, do not remove all capability. Give the agent enough access to create the plan artifact, but remove execution-oriented tools.

Q37

Domain C: Memory, state, and execution

matching magnet

hard

A repository team is redesigning how Copilot agents retain information after several incidents involving stale assumptions and repeated work. Match each requirement to the most appropriate memory or state mechanism.

The team uses Copilot cloud agent, Copilot code review, and Copilot CLI. Some information should help future repository work, while other information is only useful during one task or must be retained permanently for audit purposes.

Correct matches:

A coding convention discovered during a Copilot cloud agent session should later help Copilot code review in the same repository. → **Repository-level Copilot Memory fact scoped to the same repository**

A user's preferred command-line explanation style should apply only to that user's later Copilot CLI interactions. → **User-level Copilot Memory preference scoped to the initiating user**

A regulatory decision about an API migration must survive longer than Copilot Memory retention limits. → **External durable memory such as a committed state file or system of record**

The agent needs to remember the files it has opened while completing one uninterrupted refactoring session. → **Short-term context window for the active session**

Choices: Session log only, used after completion for human inspection, External durable memory such as a committed state file or system of record, User-level Copilot Memory preference scoped to the initiating user, Short-term context window for the active session, Repository-level Copilot Memory fact scoped to the same repository, Organization-wide repository memory shared across unrelated repositories

✓ **Why the correct answer is right**

Repository-level Copilot Memory stores repository facts such as coding conventions and can be used by supported Copilot features in that repository. User-level preferences are tied to the individual user. Decisions that must persist beyond Copilot Memory's retention behavior belong in an external durable artifact. Active-session working context is short-term memory and should not be treated as durable state.

💡 For GH-600 memory questions, first identify scope and lifespan. Repository facts, user preferences, task-local context, and permanent external records solve different problems.

Q38**Domain A: Agent architecture and SDLC integration****matching magnet****hard**

An organization is introducing custom Copilot agents for architecture planning, documentation cleanup, and test-maintenance tasks. The platform team wants to make the catalog usable across teams while preventing accidental invocation of a planning-only agent during ordinary coding tasks. Match each design decision to its primary architectural effect.

The team is using the organization-level custom agents repository pattern and agent profiles written with YAML front matter followed by Markdown instructions.

Correct matches:

Create the custom agent repository from GitHub's template, name it .github-private, and choose Internal visibility → **Publishes a governed organization catalog readable by members without manual per-user sharing**

Place planning-agent.agent.md under the custom agents file structure with YAML front matter and Markdown instructions → **Defines a reusable specialized agent profile that Copilot can load from the catalog**

Set disable-model-invocation: true while leaving the planning agent user-invocable → **Prevents automatic context-based use while still allowing intentional human selection**

Write the planning agent prompt to produce headings for assumptions, steps, risks, and acceptance criteria → **Makes the agent produce a structured planning artifact suitable for review**

Choices: Publishes a governed organization catalog readable by members without manual per-user sharing, Defines a reusable specialized agent profile that Copilot can load from the catalog, Prevents automatic context-based use while still allowing intentional human selection, Makes the agent produce a structured planning artifact suitable for review, Grants broad MCP tool access for every repository in the organization, Stores cross-session repository memory for future unrelated tasks

 Why the correct answer is right

The .github-private internal repository is the organization-level home for custom agents. Agent profiles use YAML front matter plus Markdown instructions. The current front-matter field disable-model-invocation controls automatic use, and structured prompt requirements make planning output inspectable in standard review workflows.

 For custom-agent architecture questions, distinguish catalog placement, invocation behavior, and output shape. Avoid choosing tool-permission or memory explanations unless the scenario is primarily about those controls.

custom-agent-architecture

agent-catalog

manual-selection

specialized-agents

Q39**Domain A: Agent architecture and SDLC integration**

matching magnet

hard

A platform enablement team is standardizing how several repositories delegate routine fixes to Copilot cloud agent. The team wants each artifact in the workflow to have a clear role in the agent lifecycle instead of treating every artifact as general documentation. Match each artifact to its primary SDLC purpose.

The repositories use GitHub issues, agent-created branches, pull requests, status checks, and session logs as the normal collaboration surface for agent work.

Correct matches:

Issue template fields that require affected paths, constraints, and acceptance criteria before assignment → **Defines the task input contract before the agent starts work**

Draft pull request body that lists proposed changes, assumptions, risks, and test strategy → **Provides an inspectable planning artifact before implementation is accepted**

Verified Copilot-authored commit with a link to the related agent session log → **Creates traceability from repository history back to the agent run**

Required status check that must pass before changes from copilot/* branches can merge → **Acts as an evaluation gate inside the normal PR workflow**

Choices: Defines the task input contract before the agent starts work, Provides an inspectable planning artifact before implementation is accepted, Creates traceability from repository history back to the agent run, Acts as an evaluation gate inside the normal PR workflow, Scopes external network access for the runtime environment, Persists long-term memory across unrelated repository sessions

✓ **Why the correct answer is right**

A well-architected agent workflow separates task definition, planning, action, and evaluation. Required issue fields define inputs and success criteria; the draft PR body is a reviewable plan; verified commits and session logs preserve traceability; required checks evaluate agent output before merge.

💡 For GH-600 Domain A matching items, map each artifact to its lifecycle role. Do not confuse SDLC control-plane artifacts with runtime controls such as firewall rules or memory settings.

sdlc-integration

plan-act-evaluate

inspectable-artifacts

github-control-plane

Q40**Domain C: Memory, state, and execution**

code or config artifact

hard

A team adds a lightweight GitHub Actions check to prevent an agent from resuming work from an outdated checkpoint. Review the workflow artifact. What change best improves continuity without turning the check into a general CI gate?

The agent writes a JSON checkpoint to `.github/agent-state/current-task.json` in its working branch. The checkpoint includes the issue number, last processed branch HEAD, and acceptance-criteria hash.

 `.github/workflows/agent-state-continuity.yml`

```
name: Agent state continuity
on:
  pull_request:
    types: [opened, synchronize, edited]

jobs:
  check-agent-state:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Read checkpoint
        run: |
          test -f .github/agent-state/current-task.json
          cat .github/agent-state/current-task.json
      - name: Validate issue binding only
        run: |
          ISSUE=$(jq -r '.issue' .github/agent-state/current-task.json)
          test "$ISSUE" != "null"
```

- A** Replace the checkpoint file with a full transcript so every prior command remains available.
- B** Run the entire unit-test suite in this job before allowing any checkpoint to be read.
- C** Remove the workflow and trust Copilot Memory validation to detect all resume-state problems.
- D** Also compare stored branch HEAD and acceptance-criteria hash to current PR state before resuming. ✓

✓ **Why the correct answer is right**

The workflow currently verifies only that a checkpoint exists and is linked to an issue. To prevent stale context and drift, the resume check should validate that the checkpoint corresponds to the current branch HEAD and original acceptance criteria before the agent continues.

✗ **Why the other options are wrong**

- **A:** A full transcript adds context noise and may still omit a concise statement of current progress, branch state, and decisions.

- **C:** Copilot Memory validates repository facts, but a task checkpoint is execution state that should be checked separately.
- **B:** Unit tests are useful quality signals, but they do not prove the agent is resuming from the correct checkpoint.

💡 In artifact questions, identify what the config already checks, then add the narrow missing state validation rather than replacing it with broad logging or unrelated CI.

state-artifact

stale-context-prevention

workflow-gate

Q41

Domain B: MCP/tool use and environment permissions

multi select

hard

A Windows-only repository needs Copilot cloud agent to build with an internal compiler installed on self-hosted runners. The security team requires controlled outbound internet access through the corporate proxy. Select TWO configuration actions the platform team should take.

The repository cannot build on the default Ubuntu runner, and the organization already manages ephemeral self-hosted Windows runners.

- A Use a macOS self-hosted runner label because it has stronger isolation for agent tasks.
- B Configure proxy environment variables as Agents variables, Agents secrets, or runner settings. ✓**
- C Keep the integrated cloud agent firewall enabled to enforce outbound controls on Windows.
- D Disable the integrated firewall and enforce network controls on the self-hosted runner network. ✓**
- E Store proxy credentials in Dependabot secrets so all automation shares the same proxy.

✓ **Why the correct answer is right**

Copilot cloud agent supports Ubuntu x64 and Windows 64-bit self-hosted runners, but its integrated firewall is not compatible with self-hosted runners. For proxy use, GitHub documents proxy environment variables that can be set through Agents variables, Agents secrets, or directly on the runner.

✗ **Why the other options are wrong**

- **C:** The integrated firewall is not compatible with self-hosted runners and would block use of the cloud agent.
- **A:** Copilot cloud agent does not support macOS runners for this use case.
- **E:** Copilot cloud agent does not read Dependabot secrets, and proxy values should be provided through supported agent or runner mechanisms.

💡 Runner-environment questions test compatibility boundaries. Look for the supported OS, firewall compatibility, and where environment variables actually come from.

self-hosted-runners

network-controls

proxy-configuration

Q42**Domain E: Multi-agent orchestration and incident response**

single choice

hard

Two specialized agents propose different architectural fixes for the same incident. One changes retry behavior in the API gateway, while the other changes idempotency handling in the worker. Both PRs pass their isolated tests, but applying both would duplicate retries and increase load. What should the orchestrator do next?

The incident commander needs a defensible recovery decision and an audit trail explaining why one approach was accepted.

- A** Escalate both PRs for human comparison, accept one approach, and document the rejected alternative. ✓
- B** Select the smaller diff automatically to minimize review time during the incident.
- C** Merge both PRs because each passed its isolated checks and addresses the outage symptom.
- D** Ask both agents to rewrite their patches until Git can merge the branches cleanly.

✓ **Why the correct answer is right**

Contradictory outputs require an explicit decision, not just a clean merge. Human comparison of the competing PRs preserves auditability and prevents incompatible architectural decisions from being combined.

✗ **Why the other options are wrong**

- **C:** Passing isolated checks does not prove the combined behavior is safe when the fixes duplicate retry semantics.
- **D:** A clean Git merge does not resolve the higher-level architectural contradiction between the two approaches.
- **B:** Diff size is a weak proxy for correctness and does not explain the incident recovery decision.

💡 GH-600 conflict questions may describe tests passing. If the conflict is semantic or architectural, the correct answer resolves the decision explicitly rather than relying only on merge mechanics.

contradictory-outputs

human-in-the-loop-recovery

multi-agent-conflict

Q43**Domain C: Memory, state, and execution**

multi select

very_hard

A repository owner reviews Copilot Memory after several abandoned pull requests and finds facts that may no longer match the codebase. The team wants to reduce stale influence while preserving useful validated knowledge. Select TWO actions that best align with Copilot Memory behavior.

Some facts were learned from pull requests that were closed without merging. Other facts are still supported by files on the default branch.

- A Rewrite every stale fact into repository custom instructions to make its source explicit.
- B Manually delete repository-level facts that are misleading, inappropriate, or no longer useful. ✓**
- C Disable Copilot Memory for the repository so existing facts stop being created for that repo only.
- D Keep all learned facts indefinitely because closed pull requests cannot affect future Copilot behavior.
- E Rely on citation validation and 28-day unused-entry deletion to limit stale memory over time. ✓**

✓ **Why the correct answer is right**

Repository owners can review and delete repository-level facts they consider inappropriate, misleading, or incorrect. Copilot validates facts against the current codebase before using them, and stored facts or preferences that go unused are deleted after 28 days.

✗ **Why the other options are wrong**

- **D:** Facts may be captured from closed pull requests, so the correct safeguard is validation against current code, not assuming no future effect.
- **C:** Copilot Memory is enabled per user or by organization and enterprise policy, not as a per-repository toggle that only stops one repo.
- **A:** Moving stale facts into custom instructions can make outdated guidance more persistent and bypass the memory validation model.

💡 When memory may be stale, prefer documented curation and validation mechanisms. Be cautious of answers that make stale information more permanent.

memory-expiration

memory-reset

curation

Q44

Domain F: Guardrails, safety, accountability, and governance

code or config artifact

hard

An internal audit finds that Copilot-related behavior can be changed by editing repository configuration files. The team wants edits to Copilot instructions and setup behavior to require the AI governance team's review. Which CODEOWNERS change best satisfies the requirement without over-gating unrelated files?

The repository already has a ruleset requiring review from Code Owners before protected-branch merges.

 **.github/CODEOWNERS**

```
# Existing ownership rules
*.md @octo-org/docs
/.github/workflows/deploy.yml @octo-org/devops
/src/security/** @octo-org/appsec
```

- A Add /.github/** owned by every maintainer team to reduce approval bottlenecks.
- B Add /README.md and /docs/** owned by AI governance to cover Copilot guidance.
- C Add /.github/workflows/* @octo-org/ai-governance and remove current DevOps ownership for deploy workflows.
- D Add /.github/copilot-instructions.md and /.github/workflows/copilot-setup-steps.yml owned by AI governance.** ✓

✓ **Why the correct answer is right**

GitHub recommends protecting important Copilot and MCP configuration files with CODEOWNERS and requiring Code Owner review. Targeting the Copilot instruction and setup workflow files enforces governance review without broadly blocking unrelated GitHub configuration changes.

✗ **Why the other options are wrong**

- **C:** This over-gates all workflows and removes the existing specialist owner for deployment workflow changes.
- **B:** Documentation ownership does not protect the files that directly affect Copilot behavior and setup execution.
- **A:** Assigning broad ownership to every maintainer weakens accountability and over-applies review requirements.

💡 For governance questions involving CODEOWNERS, prefer narrow ownership of high-impact agent configuration files over broad patterns that create noise or dilute accountability.

codeowners

configuration-governance

change-control

Q45**Domain D: Evaluation, telemetry, error analysis, and tuning**

single choice

medium

A payments team asks Copilot cloud agent to add missing unit tests for a legacy refund module. The agent opens pull requests quickly, but reviewers say the tests often assert implementation details rather than business behavior. Which evaluation approach best aligns with the team's development intent?

The team wants to increase confidence in refund behavior without rewarding agents for adding brittle tests that inflate coverage only.

- A Count Copilot sessions per developer and compare adoption trends across repositories after the pilot.
- B Track test pass rate, coverage delta, and reviewer feedback about behavior coverage versus implementation coupling. ✓**
- C Use median time to open the pull request as the primary success signal for the test-writing task.
- D Measure the number of test files created and the total lines of test code added per pull request.

✓ **Why the correct answer is right**

The correct evaluation combines quantitative signals, such as pass rate and coverage delta, with qualitative review feedback that reflects whether the generated tests match the intended behavior-focused outcome.

✗ **Why the other options are wrong**

- **D:** File and line counts are easy to collect but encourage volume instead of useful behavioral coverage.
- **C:** Pull request speed is useful operational telemetry, but it does not show whether tests improve confidence.
- **A:** Adoption metrics help program reporting, but they do not evaluate the quality of this agent task.

💡 For GH-600 evaluation questions, prefer signals that measure the intended outcome and constraints, not only activity volume or adoption.

evaluation-signals

development-intent-alignment

qualitative-metrics

Q46

Domain F: Guardrails, safety, accountability, and governance

matching magnet

hard

A security team is reviewing three proposed Copilot cloud agent tasks before expanding agent use into compliance-sensitive repositories. Match each observed risk to the most appropriate guardrail or accountability response.

The team wants to preserve normal coding velocity but prevent agent behavior that could bypass human review, expose secrets, or weaken the review path.

Correct matches:

An issue contains a normal bug-fix request plus an HTML comment telling the agent to ignore tests and disclose source snippets. → **Rely on hidden-character filtering for the HTML comment, review the visible request, and retain the session record for audit.**

A Copilot-authored pull request adds a GitHub Actions workflow and the merge box prompts for workflow approval. → **Inspect the workflow changes before selecting Approve and run workflows; keep the approval gate enabled for agent PRs.**

The agent needs a private package-registry token, and a developer proposes reusing an existing Actions secret. → **Create a narrowly scoped Agents secret for the registry and keep Actions secrets outside the agent environment.**

Choices: Enable Copilot content exclusions for the repository and assume the cloud agent will follow that exclusion policy., Rely on hidden-character filtering for the HTML comment, review the visible request, and retain the session record for audit., Inspect the workflow changes before selecting Approve and run workflows; keep the approval gate enabled for agent PRs., Create a narrowly scoped Agents secret for the registry and keep Actions secrets outside the agent environment., Disable built-in security validation temporarily so the agent can complete the pull request before policy review.

✓ **Why the correct answer is right**

GitHub documents that hidden content such as HTML comments is filtered before being passed to Copilot cloud agent, while session logs and audit artifacts support accountability. Agent-created pull requests have Actions workflow runs blocked by default until a user with write access approves them, which is especially important when workflow files change. Copilot cloud agent uses dedicated Agents secrets and variables; GitHub Actions, Codespaces, and Dependabot secrets are not passed to the agent.

💡 For GH-600 guardrail questions, separate controls by boundary: prompt-input filtering, workflow execution approval, and secret exposure are different controls even when they appear in the same pull request.

guardrail-selection

workflow-approval

secrets-boundaries

prompt-injection-mitigation

Q47**Domain C: Memory, state, and execution**

single choice

medium

A team maintains a repository where Copilot frequently needs the same project-specific build command and database access pattern during cloud agent work and later code review. The information is stable, repository-specific, and supported by source files in the repo. Which memory strategy should the team use first?

The team wants Copilot cloud agent and Copilot code review to benefit from the same validated repository knowledge without introducing a separate storage service.

- A Create an external vector store and require every agent task to query it before coding.
- B Use Copilot Memory repository-level facts so validated repo knowledge can be reused across supported features. ✓**
- C Save the pattern as a user-level preference so code review applies each developer's style.
- D Store the pattern only in the current prompt so each session starts with fresh context.

✓ **Why the correct answer is right**

Copilot Memory is intended for repository-level facts such as coding conventions, architectural decisions, build commands, and project-specific rules. Those facts can be used by Copilot cloud agent, Copilot code review, and Copilot CLI, with code review using repository-level facts only.

✗ **Why the other options are wrong**

- **D:** Prompt-only context is short-term and disappears after the session, so the team would keep restating stable repository facts.
- **A:** An external store can be appropriate for permanent or custom memory, but it adds unnecessary complexity for supported repository facts.
- **C:** User-level preferences are private to the user and are not applied during Copilot code review.

💡 For GH-600 memory questions, match the scope first: repo facts belong in repository-level memory, personal style in user preferences, and custom durable stores only when built-in memory is insufficient.

memory-strategy-selection

copilot-memory

external-memory

Q48**Domain E: Multi-agent orchestration and incident response**

multi select

hard

An operations dashboard shows parent Copilot SDK sessions as successful even when a delegated incident-investigation sub-agent fails. The team needs the dashboard to expose failed, partial, and stalled sub-agent executions. Select THREE implementation changes the team should make.

The dashboard is used during live incidents and during post-hoc review meetings.

- A Suppress failed sub-agent events unless the final parent response also contains an error.
- B Group event records by agent description so related incident skills appear together.
- C Treat the parent response text as the single source of truth for all sub-agent outcomes.
- D Track elapsed time since each started event to flag agents that have not completed. ✓
- E Persist toolCallId, agentName, status, timestamps, and error details for later analysis. ✓
- F Subscribe to subagent.started, subagent.completed, and subagent.failed events from the parent session. ✓

✓ **Why the correct answer is right**

The parent session emits sub-agent lifecycle events, including started, completed, and failed events. Tracking time since started exposes stalls, and persisting toolCallId, identity, status, timestamps, and errors supports later reconstruction and audit.

✗ **Why the other options are wrong**

- **C:** The parent response can summarize outcomes, but relying on prose loses structured failure and timing signals.
- **A:** Suppressing sub-agent failures hides degraded behavior that the parent may partially recover from or fail to mention.
- **B:** Descriptions help routing, but they are not reliable correlation keys for execution status or incident forensics.

💡 When an item asks about degraded multi-agent behavior, prefer structured lifecycle telemetry over final natural-language summaries. Completion, failure, and timing signals matter.

subagent-failures

degraded-behavior

operational-observability

Q49**Domain D: Evaluation, telemetry, error analysis, and tuning**

multi select

hard

A retail platform reviews three failed Copilot cloud agent sessions for the same bug fix. In each session, the plan identified the correct service, but the commits changed a deprecated helper and the tests failed because the agent ran a package script from the wrong workspace. Select TWO root causes that best explain the failures.

The session logs show successful repository search calls, followed by shell commands executed from the repository root instead of the service directory.

- A Evaluation gap, because the team measured merge rate instead of total token usage.
- B Reasoning error, because the agent chose the wrong service during its initial planning step.
- C Prompt issue, because the user did not specify every test command in the initial issue description.
- D Tool misuse, because the agent invoked the test command from the wrong working directory. ✓**
- E Context issue, because the agent edited deprecated code despite finding the correct active service. ✓**

✓ **Why the correct answer is right**

The wrong working directory is tool misuse because the command was invoked with the wrong execution context. Editing deprecated helper code after identifying the correct service indicates a context issue in how repository information was applied.

✗ **Why the other options are wrong**

- **C:** A prompt can be improved, but the logs show a concrete command-location problem and stale code selection.
- **B:** The plan found the correct service, so the initial planning step was not the primary failure.
- **A:** Merge rate and token usage may be poor evaluation choices, but they do not explain this execution failure.

💡 When a scenario includes logs or command history, classify the observed failure directly before proposing broader prompt or metric changes.

failure-analysis

session-logs

root-cause-classification

Q50

Domain B: MCP/tool use and environment permissions

single choice

medium

A repository team is creating a custom Copilot agent that answers architecture questions by inspecting source files and README content. The agent must not modify files, run commands, or call external systems. Which tool configuration best meets the requirement?

The agent will be available to developers during design reviews and must follow least-privilege access.

- A Set tools to an empty list and rely on prompt instructions for repository access.
- B Set tools to execute, read, and search so the agent can verify findings.
- C Omit the tools property so the agent can decide which built-in tools are needed.
- D Set tools to read and search aliases only, such as read and search.** ✓

✓ **Why the correct answer is right**

A read-only inspection agent needs only file reading and search capabilities. GitHub custom agent configuration supports limiting tools to specific aliases, while omitting tools enables all available tools.

✗ **Why the other options are wrong**

- **C:** Omitting tools grants all available tools, which exceeds the read-only requirement.
- **B:** Adding execute enables shell access, which is unnecessary for answering architecture questions.
- **A:** An empty tools list disables all tools, preventing the agent from inspecting repository content.

💡 For GH-600 tool-scope questions, map the task to the minimum concrete tool set. Prompt instructions are not a substitute for configured tool restrictions.

tool-selection

least-privilege

custom-agent-tools

Q51**Domain D: Evaluation, telemetry, error analysis, and tuning****code or config artifact****very_hard**

A repository team pilots a custom agent for small accessibility fixes. After four sessions, reviewers find the agent frequently edits unrelated styling files and invokes external package managers even when the requested change is in one component. How should the agent profile be tuned?

The team wants to reduce unrelated diffs while preserving the agent's ability to inspect, edit, and verify component-level accessibility changes.

 .github/agents/a11y-fixer.agent.md

```
---
name: ally-fixer
description: Fixes accessibility defects in UI components
target: github-copilot
tools: ["*"]
---
```

You fix accessibility issues quickly. Explore the repository, update the code, run whatever commands seem useful, and prepare the pull request for review.

- A Add a larger prompt describing WCAG guidance but keep the current unrestricted tool list.
- B Restrict tools to needed aliases and add instructions for scoped edits plus targeted validation. ✓**
- C Remove edit access so the agent can only produce a written accessibility remediation plan.
- D Keep all tools enabled and ask reviewers to downvote comments on unrelated styling changes.

 Why the correct answer is right

The evidence points to tool overuse and vague constraints. A tuned profile should limit the tool surface to the task and add explicit instructions for scoped edits and validation.

 Why the other options are wrong

- **D:** Feedback after the fact does not reduce the repeated unnecessary tool use or unrelated diffs.
- **C:** Removing edit access changes the agent's purpose from fixer to planner and blocks useful work.
- **A:** More domain guidance may improve accessibility reasoning but leaves the tool misuse pattern unaddressed.

 If the artifact shows tools set to a broad wildcard and the failure is unrelated activity, narrow tools and add execution constraints.

Q52

Domain C: Memory, state, and execution

single choice

very_hard

A Copilot cloud agent session learned that service clients must be created through `src/platform/clientFactory.ts`. A later pull request changes the factory pattern on a feature branch. Before applying the old fact during new work on that branch, what behavior best prevents stale context?

The team wants repository knowledge to persist when it is still true, but not override code that has legitimately changed on the current branch.

- A Copy the stored fact into custom instructions so it survives branch-level code changes.
- B Delete all repository-level memory whenever a branch changes the referenced files.
- C Validate the fact's code citations against the current branch before using the memory. ✓**
- D Apply the stored fact first, then let CI determine whether the pattern still compiles.

✓ **Why the correct answer is right**

Repository-level facts in Copilot Memory are stored with citations to supporting code. When a fact is relevant, Copilot checks those citations against the current branch and uses only validated facts.

✗ **Why the other options are wrong**

- **D:** Relying on CI after applying stale context can cause unnecessary or misleading changes before validation occurs.
- **A:** Custom instructions are not a substitute for citation validation and may preserve outdated guidance after code changes.
- **B:** Deleting all memory on any referenced-file change is excessive because validation can distinguish current facts from stale ones.

💡 For stale-memory scenarios, look for validation against the current branch or current codebase, not broad deletion or post-hoc detection.

Q53**Domain C: Memory, state, and execution****sequence order****very_hard**

A Copilot cloud agent session was interrupted during a multi-step refactor. When the agent resumes, reviewers want it to avoid redoing completed work and to detect whether its current plan has drifted from the original issue. In which order should the recovery workflow run?

The repository stores progress checkpoints in the pull request description and a committed JSON state file. Human reviewers added steering comments after the first failed attempt.

Correct order:

1. Read the original issue, acceptance criteria, and reviewer steering comments defining the intended outcome.
2. Fetch branch HEAD and load the latest durable checkpoint from the PR or state file.
3. Compare the current diff and in-progress plan against the original requirements to identify drift.
4. Prune irrelevant tool history, re-anchor on the spec, and restart from the valid checkpoint if needed.
5. Update the PR checklist or state file after the next validated unit of work completes.

✓ **Why the correct answer is right**

The agent should first re-establish intent from the issue and human steering, then load the latest branch and durable checkpoint. Only after comparing current work against the original requirements can it determine whether drift occurred. If drift exists, it should re-anchor and resume from the last valid checkpoint, then persist the next completed state update.

✗ **Why the other options are wrong**

- **A:** Writing a new checkpoint before validating intent and branch state can persist drift or duplicate incomplete work.
- **C:** Drift comparison is useful only after both the original requirements and current branch state are loaded.
- **D:** Loading branch state before reading the source requirements gives the agent state without a correctness baseline.
- **E:** Pruning and restart decisions should follow drift detection, not precede the comparison.

💡 Sequence questions in this domain reward recovery flows that re-read intent, load durable state, validate for drift, then resume. Avoid options that persist state too early.

state-resume

context-drift

durable-checkpoints

Q54

Domain B: MCP/tool use and environment permissions

sequence order

very_hard

A repository team needs Copilot cloud agent to consult a vendor incident system during bug-fix tasks. The vendor exposes a remote MCP endpoint and supports PAT-based authorization. Put the configuration steps in the safest order.

The agent should read incident summaries and links, but it must not acknowledge incidents, mutate ticket state, or use unrelated vendor tools. The team wants to validate the integration without exposing broader repository or vendor access.

Correct order:

1. Identify the exact read-only MCP tools, transport type, endpoint URL, and minimum PAT permissions.
2. Create a repository-level Agents secret for the vendor PAT using a COPILOT_MCP_ prefixed name.
3. Add repository MCP JSON with explicit tools, URL, type, and Authorization header substitution.
4. Save and validate the configuration, then run a low-risk task and inspect the session evidence.

✓ **Why the correct answer is right**

The safest sequence starts by defining the required tools and minimum permissions, then provisioning the secret in the Agents secret store with the MCP prefix, then adding a repository MCP configuration that allowlists only those tools and uses header substitution. Validation should occur after the configuration exists and should use a low-risk task with session evidence review.

✗ **Why the other options are wrong**

- **A:** Creating the secret before defining required scopes can lead to over-permissioned PATs or naming that does not match the final MCP configuration.
- **B:** This is the first step because least-privilege tool and token design should precede secret creation and JSON configuration.
- **C:** Adding MCP JSON before the secret and permission design risks validation failures or broad tools such as an asterisk wildcard.
- **D:** Validation is necessary, but it belongs after the MCP server and secret configuration are in place.

💡 On sequence questions, put discovery and least-privilege design before implementation. Then configure secrets, wire the MCP JSON, and validate with traceable evidence.

mcp-server-setup

remote-mcp-authentication

tool-allowlisting

safe-validation

Q55**Domain A: Agent architecture and SDLC integration**

multi select

very_hard

A regulated engineering group is piloting Copilot cloud agent for non-production repositories. The architecture review board wants traceable agent work without introducing a separate observability product during the pilot. Select TWO GitHub-native design choices that best satisfy this requirement.

The pilot scope is SDLC workflow design and traceability. Network controls, audit-log retention policy, and compliance approval rules are handled by other teams.

- A Disable draft pull requests so agent changes appear immediately in the standard review queue.
- B Use Copilot session logs to inspect progress, tool use, validation, token usage, and session length. ✓**
- C Replace agent-created commits with squashed human-authored commits before reviewers inspect the branch.
- D Require developers to paste local terminal transcripts into the PR after each review.
- E Export agent summaries into a wiki page and review the wiki during sprint demos.
- F Preserve Copilot-authored verified commits with the initiating user as co-author and session links. ✓**

✓ **Why the correct answer is right**

Session logs provide GitHub-native visibility into the agent's work, tools, and validation. Copilot-authored verified commits with co-author attribution and session-log links preserve traceability from code changes back to the agent session.

✗ **Why the other options are wrong**

- **E:** A wiki summary is secondary documentation and can drift from the actual agent session and commits.
- **D:** Local terminal transcripts are manual, inconsistent, and not the native source of truth for cloud agent work.
- **A:** Disabling drafts reduces the explicit review staging model rather than improving traceability.
- **C:** Replacing agent-authored commits removes attribution that GitHub provides for Copilot changes.

💡 When GH-600 asks for observability in Domain A, prefer existing GitHub artifacts such as PRs, commits, checks, and session logs over copied summaries.

traceability

github-control-plane

agent-observability

Q56**Domain F: Guardrails, safety, accountability, and governance****code or config artifact****hard**

A platform engineer reviews a Copilot setup workflow for a repository that only needs to install dependencies before Copilot starts. The workflow currently grants write permissions even though the setup job only checks out code and runs `npm ci`. Which change best enforces least privilege?

The organization requires that agent execution contexts receive only the permissions needed for the task.

 `.github/workflows/copilot-setup-steps.yml`

```
name: "Copilot Setup Steps"

on:
  workflow_dispatch:
  push:
    paths:
      - .github/workflows/copilot-setup-steps.yml
  pull_request:
    paths:
      - .github/workflows/copilot-setup-steps.yml

jobs:
  copilot-setup-steps:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
      actions: write
    steps:
      - name: Checkout code
        uses: actions/checkout@v6
      - name: Install dependencies
        run: npm ci
```

- A** Change job permissions to `contents: read only` and keep the setup steps unchanged. ✓
- B** Keep permissions unchanged because Copilot receives a separate token for its repository operations.
- C** Move permissions to workflow scope and add `id-token: write` for future package access.
- D** Remove checkout and grant `packages: write` to avoid repository reads during setup.

✓ **Why the correct answer is right**

The setup job only needs to clone the repository and install dependencies. GitHub's Copilot setup guidance recommends using the lowest permissions possible and notes that checkout-style setup requires `contents: read`.

✗ Why the other options are wrong

- **B:** The separate Copilot token does not justify over-permissioning the GITHUB_TOKEN used by setup steps.
- **C:** Adding id-token: write expands the trust boundary without a current requirement for OIDC authentication.
- **D:** packages: write is unnecessary for installing dependencies and increases the impact of a compromised setup step.

💡 When a GH-600 question shows a setup workflow, inspect what the job actually does. Least privilege usually means reducing GITHUB_TOKEN permissions, not relying on downstream reviews.

least-privilege

copilot-setup-steps

github-token-permissions

Q57

Domain B: MCP/tool use and environment permissions

single choice

hard

A team connects Copilot cloud agent to a remote MCP server for its incident platform. The server returns 401 with an OAuth browser-consent challenge, and the agent cannot complete the handshake. What should the team do?

The incident platform also supports API-token authentication through an HTTP Authorization header.

- A** Switch the MCP server to local stdio without changing the authentication method.
- B** Configure the remote MCP server with token-based headers using an Agents secret. ✓
- C** Wait for the cloud agent to open the OAuth consent page during the next run.
- D** Move the OAuth client secret into GitHub Actions secrets and retry the server.

✓ Why the correct answer is right

Copilot cloud agent currently does not support remote MCP servers that rely on OAuth. For remote HTTP or SSE MCP servers, headers can reference Copilot Agents secrets or variables, which should use the COPILOT_MCP_ prefix when used by MCP configuration.

✗ Why the other options are wrong

- **C:** The cloud agent does not complete a remote MCP OAuth browser-consent flow.
- **D:** Copilot cloud agent cannot read Actions secrets for its MCP configuration.
- **A:** Changing transport alone does not solve an OAuth flow that the cloud agent cannot complete.

💡 When an MCP authentication answer mentions Actions secrets or unattended OAuth for cloud agent, check current GitHub docs before trusting it.

mcp-authentication

remote-mcp

agent-secrets

Q58**Domain D: Evaluation, telemetry, error analysis, and tuning**

multi select

hard

A developer says Copilot cloud agent completed a refactor, but the pull request has no CI results and the agent's final comment says validation was inconclusive. The team needs evidence for a failure review. Select TWO artifacts they should inspect first.

The repository normally runs linting and unit tests through GitHub Actions, but workflows did not appear on this pull request.

- A The organization's monthly Copilot seat assignment export and license activity report.
- B The repository star history and traffic graph for the week the pull request was opened.
- C The pull request timeline and workflow area showing whether Actions runs were approved or skipped. ✓
- D The Copilot session log showing tool use, validation attempts, and any reported blockers. ✓
- E The developer's local IDE suggestion acceptance metrics for files touched by the refactor.

✓ **Why the correct answer is right**

Session logs reveal how the agent approached validation, including tools used and blockers. The pull request workflow area and timeline show whether Actions ran, were skipped, or required approval.

✗ **Why the other options are wrong**

- **A:** License data helps entitlement and adoption analysis, not this pull request's validation failure.
- **B:** Repository popularity telemetry is unrelated to CI execution or agent validation evidence.
- **E:** Local IDE acceptance metrics do not explain cloud agent workflow behavior on the pull request.

💡 For failure-review questions, prioritize artifacts closest to the execution path: session logs, pull request timeline, workflow runs, and job artifacts.

workflow-artifacts

logs

failure-evidence

Q59

Domain B: MCP/tool use and environment permissions

matching magnet

very_hard

A regulated engineering group is moving Copilot cloud agent from a lab repository into several service repositories with private dependencies and custom runners. Match each environment constraint to the best remediation.

The organization wants deterministic builds while preserving data-exfiltration controls. Teams are allowed to use self-hosted runners only when network controls are owned by the platform team.

Correct matches:

A setup step runs npm ci against a private package registry and needs a registry token available to the shell command. → **Create an Agents secret or variable for the token so setup steps can consume it as an environment variable.**

Copilot reports that a dependency download host is blocked while running on the default GitHub-hosted cloud agent environment. → **Add only the required host to the Copilot cloud agent firewall allowlist or approved network path.**

A Windows-only build requires Copilot cloud agent to use a self-hosted Windows runner inside the corporate network. → **Use an appropriate Windows runner strategy and implement runner-level network controls because the integrated firewall is not compatible there.**

A repository setup workflow adds a job named prepare-copilot, but Copilot does not run the setup steps before starting. → **Rename the job to copilot-setup-steps in .github/workflows/copilot-setup-steps.yml on the default branch.**

Choices: Create an Agents secret or variable for the token so setup steps can consume it as an environment variable., Add only the required host to the Copilot cloud agent firewall allowlist or approved network path., Use an appropriate Windows runner strategy and implement runner-level network controls because the integrated firewall is not compatible there., Rename the job to copilot-setup-steps in .github/workflows/copilot-setup-steps.yml on the default branch., Disable the firewall for all repositories so the dependency manager can resolve future hosts., Move the token into a Dependabot or Actions secret because Copilot automatically inherits those values.

✓ Why the correct answer is right

Agents secrets and variables are the correct mechanism for values that Copilot cloud agent and setup steps need. The default cloud agent environment has a firewall with an allowlist model, so a blocked dependency host should be added narrowly rather than disabling controls broadly. Windows and self-hosted runner scenarios require runner-level network controls because the integrated firewall is not compatible. Copilot setup steps must be defined in the special workflow file on the default branch and the job must be named copilot-setup-steps.

💡 Environment-permission questions usually test the narrowest control that restores execution. Prefer Agents secrets, precise firewall entries, supported runner constraints, and exact setup-step conventions over broad disablement.

execution-environment

firewall

agent-secrets

runner-networking

Q60**Domain B: MCP/tool use and environment permissions****single choice****hard** **Case study — Case Study: SkyHarbor Robotics Agentic Maintenance Platform**

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

SkyHarbor Robotics operates autonomous inspection drones for port authorities. The engineering organization uses GitHub Enterprise Cloud and is piloting Copilot cloud agent to repair non-flight-critical maintenance software. The pilot focuses on a single repository, skyharbor/maintenance-console, which contains a TypeScript dashboard, Python simulation tests, and infrastructure templates for non-production environments.

Existing Environment

The repository already has Copilot cloud agent enabled. A custom agent named maintenance-triage is stored under .github/agents/maintenance-triage.agent.md. The agent needs to read repository files, run tests, and call an internal FleetOps MCP server that exposes drone status and maintenance-ticket tools. Developers also have a working VS Code MCP configuration for Sentry-based issue triage, but it uses inputs and a local envFile. The platform team wants to adapt the configuration for Copilot cloud agent without copying workstation-specific assumptions.

Business Requirements

The team must reduce mean time to prepare non-production maintenance-console fixes while preserving a controlled execution path. The agent must be able to create draft pull requests with code changes, validate changes with simulation tests, and link relevant FleetOps or Sentry context in the pull request description.

Technical Requirements

The FleetOps MCP server is a remote HTTP server. It supports bearer-token authentication but does not provide a Copilot-compatible remote OAuth flow. The simulation tests call `https://sim.skyharbor.example/api/v2` from shell commands executed by the agent. The repository uses a private npm registry and a Python package mirror, both reachable through company-controlled hostnames. MCP tools must be enabled explicitly instead of exposing every available tool from a server.

Security and Compliance Requirements

Secrets used by MCP servers must not be available to arbitrary setup scripts or general shell commands unless specifically required. MCP access should be restricted to approved servers through organization-level controls where possible. Network access should be limited to the minimum domains or URL paths required for agent-driven validation. The team must be able to confirm from Copilot session logs whether MCP servers started and which tools were exposed.

SkyHarbor's maintenance-triage agent fails to start the FleetOps MCP server because the Authorization header resolves to an empty bearer token. The FleetOps token currently

exists as a repository Actions secret named FLEETOPS_TOKEN. What should the platform team do?

The FleetOps server is a remote HTTP MCP server. The team wants the token usable by the MCP server, but not broadly exposed to unrelated setup scripts.

 FleetOps MCP configuration draft

```
{
  "mcpServers": {
    "fleetops": {
      "type": "http",
      "url": "https://mcp.ops.skyharbor.example/mcp",
      "tools": ["get_aircraft_status", "open_maintenance_ticket"],
      "headers": {
        "Authorization": "Bearer ${FLEETOPS_TOKEN}"
      }
    }
  }
}
```

- A** Create an Agents secret named COPILOT_MCP_FLEETOPS_TOKEN and reference it in the Authorization header. ✓
- B** Switch the FleetOps server to OAuth and let cloud agent complete the consent flow.
- C** Enable the FleetOps server through the MCP registry and remove the Authorization header.
- D** Move the token to an Actions environment secret and keep the MCP header reference unchanged.

✓ **Why the correct answer is right**

Copilot cloud agent uses dedicated Agents secrets and variables, and MCP configuration values intended for MCP servers must use the COPILOT_MCP_ prefix. Referencing a COPILOT_MCP_ Agents secret in the remote MCP Authorization header gives the server the token without relying on Actions secrets.

✗ **Why the other options are wrong**

- **D:** Actions secrets are not passed to Copilot cloud agent as Agents secrets, so the MCP server would still not receive the expected token.
- **C:** A registry or allowlist policy governs which MCP servers can be used; it does not replace bearer-token authentication to the server.
- **B:** Copilot cloud agent documentation states that remote MCP servers using OAuth for authentication and authorization are not currently supported.

💡 For GH-600 MCP authentication questions, first identify where the secret is consumed. MCP server credentials usually belong in Agents secrets with the COPILOT_MCP_ prefix, not in general Actions secrets.

mcp-authentication

agents-secrets

least-privilege-tooling

 Case study — Case Study: SkyHarbor Robotics Agentic Maintenance Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

SkyHarbor Robotics operates autonomous inspection drones for port authorities. The engineering organization uses GitHub Enterprise Cloud and is piloting Copilot cloud agent to repair non-flight-critical maintenance software. The pilot focuses on a single repository, skyharbor/maintenance-console, which contains a TypeScript dashboard, Python simulation tests, and infrastructure templates for non-production environments.

Existing Environment

The repository already has Copilot cloud agent enabled. A custom agent named maintenance-triage is stored under .github/agents/maintenance-triage.agent.md. The agent needs to read repository files, run tests, and call an internal FleetOps MCP server that exposes drone status and maintenance-ticket tools. Developers also have a working VS Code MCP configuration for Sentry-based issue triage, but it uses inputs and a local envFile. The platform team wants to adapt the configuration for Copilot cloud agent without copying workstation-specific assumptions.

Business Requirements

The team must reduce mean time to prepare non-production maintenance-console fixes while preserving a controlled execution path. The agent must be able to create draft pull requests with code changes, validate changes with simulation tests, and link relevant FleetOps or Sentry context in the pull request description.

Technical Requirements

The FleetOps MCP server is a remote HTTP server. It supports bearer-token authentication but does not provide a Copilot-compatible remote OAuth flow. The simulation tests call `https://sim.skyharbor.example/api/v2` from shell commands executed by the agent. The repository uses a private npm registry and a Python package mirror, both reachable through company-controlled hostnames. MCP tools must be enabled explicitly instead of exposing every available tool from a server.

Security and Compliance Requirements

Secrets used by MCP servers must not be available to arbitrary setup scripts or general shell commands unless specifically required. MCP access should be restricted to approved servers through organization-level controls where possible. Network access should be limited to the minimum domains or URL paths required for agent-driven validation. The team must be able to confirm from Copilot session logs whether MCP servers started and which tools were exposed.

SkyHarbor is finalizing its production pilot controls for maintenance-console. Match each rollout need to the control that best satisfies it.

The platform team wants preventive controls rather than relying on reviewers to notice mistakes after an agent session has run.

Correct matches:

Developers may use approved MCP servers in Copilot CLI and supported IDEs, but not ad hoc local servers. → **Configure an MCP registry URL and set MCP access to Registry only**

The cloud agent's bash-driven simulation tests must call `https://sim.skyharbor.example/api/v2`. → **Add the exact staging API URL to the cloud agent firewall custom allowlist**

The FleetOps MCP token must be available to the MCP server but not to general setup scripts. → **Store the token as an Agents secret prefixed `COPILOT_MCP_` and reference it in MCP config**

Choices: Add the exact staging API URL to the cloud agent firewall custom allowlist, Store the token as an Agents secret prefixed `COPILOT_MCP_` and reference it in MCP config, Disable the cloud agent firewall and rely on PR warnings for blocked requests, Configure an MCP registry URL and set MCP access to Registry only, Set tools to `["*"]` so the agent can discover missing tool permissions, Add the token as a repository Actions secret and reference it from `mcp-servers`

✓ **Why the correct answer is right**

Registry-only MCP access constrains developers to approved MCP servers. A firewall custom allowlist entry permits the required agent-started network call while keeping the firewall enabled. A `COPILOT_MCP_` Agents secret makes the token available to MCP configuration while keeping it separate from general agent environment variables.

💡 Matching questions often mix similar-looking controls. Separate the control planes: MCP registry policies govern server choice, the cloud agent firewall governs agent-started outbound network calls, and Agents secrets govern credential injection.

mcp-allowlist

agent-firewall

secret-scope

 Case study — Case Study: SkyHarbor Robotics Agentic Maintenance Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

SkyHarbor Robotics operates autonomous inspection drones for port authorities. The engineering organization uses GitHub Enterprise Cloud and is piloting Copilot cloud agent to repair non-flight-critical maintenance software. The pilot focuses on a single repository, skyharbor/maintenance-console, which contains a TypeScript dashboard, Python simulation tests, and infrastructure templates for non-production environments.

Existing Environment

The repository already has Copilot cloud agent enabled. A custom agent named maintenance-triage is stored under .github/agents/maintenance-triage.agent.md. The agent needs to read repository files, run tests, and call an internal FleetOps MCP server that exposes drone status and maintenance-ticket tools. Developers also have a working VS Code MCP configuration for Sentry-based issue triage, but it uses inputs and a local envFile. The platform team wants to adapt the configuration for Copilot cloud agent without copying workstation-specific assumptions.

Business Requirements

The team must reduce mean time to prepare non-production maintenance-console fixes while preserving a controlled execution path. The agent must be able to create draft pull requests with code changes, validate changes with simulation tests, and link relevant FleetOps or Sentry context in the pull request description.

Technical Requirements

The FleetOps MCP server is a remote HTTP server. It supports bearer-token authentication but does not provide a Copilot-compatible remote OAuth flow. The simulation tests call `https://sim.skyharbor.example/api/v2` from shell commands executed by the agent. The repository uses a private npm registry and a Python package mirror, both reachable through company-controlled hostnames. MCP tools must be enabled explicitly instead of exposing every available tool from a server.

Security and Compliance Requirements

Secrets used by MCP servers must not be available to arbitrary setup scripts or general shell commands unless specifically required. MCP access should be restricted to approved servers through organization-level controls where possible. Network access should be limited to the minimum domains or URL paths required for agent-driven validation. The team must be able to confirm from Copilot session logs whether MCP servers started and which tools were exposed.

SkyHarbor wants to reuse its working VS Code Sentry MCP configuration with Copilot cloud agent. The current configuration uses inputs and an envFile on developer

workstations. In which order should the platform team adapt and validate it?

The team wants a repository-level configuration that runs consistently in cloud agent sessions and exposes a narrow set of Sentry tools.

Current developer MCP notes

```
{
  "servers": {
    "sentry": {
      "command": "npx",
      "args": ["-y", "@sentry/mcp-server", "--host", "${input:sentryHost}"],
      "envFile": ".env.local",
      "inputs": [
        {"id": "sentryHost", "type": "promptString", "description": "Sentry host"}
      ]
    }
  }
}
```

Correct order:

1. Create the cloud agent MCP JSON with an `mcpServers` object and an explicit tools list.
2. Replace inputs and `envFile` usage with `env` entries that reference `COPILOT_MCP_Agents` values.
3. Add the required `COPILOT_MCP_` secrets or variables as Agents secrets or variables.
4. Validate by assigning a small issue and checking Start MCP Servers session logs for exposed tools.

Why the correct answer is right

Cloud agent MCP configuration is expressed as an `mcpServers` JSON object and should include an explicit tools list. When adapting from VS Code, inputs and `envFile` usage should be replaced with `env`-based references, the referenced `COPILOT_MCP_Agents` secrets or variables must be configured, and the final configuration should be validated from a Copilot session log by inspecting the Start MCP Servers step.

 For migration-order questions, look for platform-specific conversion steps before validation. VS Code convenience features such as inputs and `envFile` usually need to be converted for cloud agent.

mcp-configuration-migration

vs-code-to-cloud-agent

mcp-validation

Q63**Domain A: Agent architecture and SDLC integration**

multi select

very_hard

 Case study — Case Study: Northstar Transit Analytics Agentic Delivery Program

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Northstar Transit Analytics builds route-planning and ridership-forecasting software for regional rail and bus agencies. The engineering organization uses GitHub Enterprise Cloud, GitHub Actions, and GitHub Copilot Enterprise. The company is expanding its use of Copilot cloud agent beyond documentation fixes into routine service changes, test scaffolding, and low-risk refactoring tasks.

Existing Environment

Northstar has 68 repositories across four product groups. Product teams currently start agent work from issues, the agents panel on GitHub.com, and occasional pull request comments. Several teams copied custom agent profiles into individual repositories, which has caused inconsistent prompts and uneven release practices. A platform team owns a shared ``.github-private`` repository but has not yet moved organization-level agents into a formal release workflow.

Business Requirements

Northstar wants agents to reduce backlog latency without reducing code review quality. Ambiguous work items should be explored before a pull request is created, while clearly scoped work may proceed directly to a pull request. Engineering managers want agent work to remain visible inside GitHub rather than in private local chat sessions or external spreadsheets.

Technical Requirements

Agent work must produce inspectable artifacts in standard GitHub tooling, including branches, pull requests when appropriate, session logs, and commit history. Teams must be able to steer an active agent session if the approach is wrong. Organization-level custom agents must be reusable across repositories after they are tested, and repository-specific experiments must not appear as broadly available agents until released.

Security and Compliance Requirements

The compliance team requires traceability from agent-authored commits to the human who initiated the task and to the agent session that explains the change. The company prefers built-in GitHub controls and auditability over creating a separate tracking system unless a requirement cannot be met through GitHub-native artifacts.

Northstar wants to standardize reusable custom agents across its organization, but still let teams test experimental agent profiles privately before broad release. Select TWO actions the platform team should take.

The case study states that teams have copied agent profiles into repositories and that the shared ``.github-private`` repository exists but does not yet have a formal organization-level

agent release workflow.

- A** Move tested organization-level agent profiles into the root ``agents`` directory before merging the release. ✓
- B** Configure the organization ``github-private`` repository from GitHub's template with appropriate internal or private visibility. ✓
- C** Continue copying profiles into product repositories so each team can merge prompt changes independently.
- D** Release organization agents from ``github/agents`` in ``github-private`` to keep paths consistent with repositories.
- E** Publish the agents publicly so discovery works the same for all licensed Northstar developers.

✓ **Why the correct answer is right**

The organization-level setup uses a ``github-private`` repository created from GitHub's custom agents template with internal or private visibility. For release, tested profiles are moved from the test path into the root ``agents`` directory so they become available across the organization.

✗ **Why the other options are wrong**

- **C:** Copying profiles into many repositories preserves the inconsistency Northstar is trying to remove and weakens release control.
- **D:** The ``github/agents`` path in ``github-private`` is for private testing; released organization-level profiles are moved to the root ``agents`` directory.
- **E:** Public publication is unnecessary and would not match the organization's need for controlled internal reuse.

💡 On GH-600 custom-agent lifecycle items, distinguish repository-level agent paths from organization-level release paths in ``github-private``; the exam often tests that boundary.

custom-agents

organization-agent-release

sdic-standardization

Q64**Domain A: Agent architecture and SDLC integration**

single choice

hard

 Case study — Case Study: Northstar Transit Analytics Agentic Delivery Program

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Northstar Transit Analytics builds route-planning and ridership-forecasting software for regional rail and bus agencies. The engineering organization uses GitHub Enterprise Cloud, GitHub Actions, and GitHub Copilot Enterprise. The company is expanding its use of Copilot cloud agent beyond documentation fixes into routine service changes, test scaffolding, and low-risk refactoring tasks.

Existing Environment

Northstar has 68 repositories across four product groups. Product teams currently start agent work from issues, the agents panel on GitHub.com, and occasional pull request comments. Several teams copied custom agent profiles into individual repositories, which has caused inconsistent prompts and uneven release practices. A platform team owns a shared ``.github-private`` repository but has not yet moved organization-level agents into a formal release workflow.

Business Requirements

Northstar wants agents to reduce backlog latency without reducing code review quality. Ambiguous work items should be explored before a pull request is created, while clearly scoped work may proceed directly to a pull request. Engineering managers want agent work to remain visible inside GitHub rather than in private local chat sessions or external spreadsheets.

Technical Requirements

Agent work must produce inspectable artifacts in standard GitHub tooling, including branches, pull requests when appropriate, session logs, and commit history. Teams must be able to steer an active agent session if the approach is wrong. Organization-level custom agents must be reusable across repositories after they are tested, and repository-specific experiments must not appear as broadly available agents until released.

Security and Compliance Requirements

The compliance team requires traceability from agent-authored commits to the human who initiated the task and to the agent session that explains the change. The company prefers built-in GitHub controls and auditability over creating a separate tracking system unless a requirement cannot be met through GitHub-native artifacts.

Northstar's route-optimization team has backlog items where the implementation path is unclear, and premature pull requests have created review churn. The team wants Copilot cloud agent to explore the repository and propose an approach before a pull request is opened. Which workflow best satisfies the requirement?

The case study states that ambiguous work should be explored before a pull request, while agent activity must remain visible in GitHub-native tooling.

- A Ask Copilot to open a pull request immediately, then revise the branch based on reviewer feedback.
- B Route the work through local IDE chat, then paste a summary into the issue before coding starts.
- C Use the agents panel to research, plan, change a branch, iterate, then create the pull request when ready. ✓**
- D Require a manually written design document before assigning any ambiguous issue to an agent.

✓ **Why the correct answer is right**

This workflow uses Copilot cloud agent's GitHub.com flow for repository research, planning, branch changes, iteration, and creating a pull request when ready. It preserves work in GitHub artifacts while avoiding premature PR review churn.

✗ **Why the other options are wrong**

- **B:** Local IDE chat can help exploration, but the case requires GitHub-native visibility rather than private local decisions pasted later.
- **A:** Opening the pull request immediately conflicts with the requirement to explore ambiguous work before creating PR review load.
- **D:** A manual design document may add process weight and does not use the agent's built-in ability to research and plan in GitHub.

💡 For GH-600 SDLC questions, prefer the workflow that keeps agent planning and iteration inspectable in GitHub while delaying broader review until the work is ready.

sdhc-integration

planning-action-boundary

inspectable-artifacts

Q65**Domain A: Agent architecture and SDLC integration****matching magnet****hard** **Case study — Case Study: Northstar Transit Analytics Agentic Delivery Program**

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Northstar Transit Analytics builds route-planning and ridership-forecasting software for regional rail and bus agencies. The engineering organization uses GitHub Enterprise Cloud, GitHub Actions, and GitHub Copilot Enterprise. The company is expanding its use of Copilot cloud agent beyond documentation fixes into routine service changes, test scaffolding, and low-risk refactoring tasks.

Existing Environment

Northstar has 68 repositories across four product groups. Product teams currently start agent work from issues, the agents panel on GitHub.com, and occasional pull request comments. Several teams copied custom agent profiles into individual repositories, which has caused inconsistent prompts and uneven release practices. A platform team owns a shared ``.github-private`` repository but has not yet moved organization-level agents into a formal release workflow.

Business Requirements

Northstar wants agents to reduce backlog latency without reducing code review quality. Ambiguous work items should be explored before a pull request is created, while clearly scoped work may proceed directly to a pull request. Engineering managers want agent work to remain visible inside GitHub rather than in private local chat sessions or external spreadsheets.

Technical Requirements

Agent work must produce inspectable artifacts in standard GitHub tooling, including branches, pull requests when appropriate, session logs, and commit history. Teams must be able to steer an active agent session if the approach is wrong. Organization-level custom agents must be reusable across repositories after they are tested, and repository-specific experiments must not appear as broadly available agents until released.

Security and Compliance Requirements

The compliance team requires traceability from agent-authored commits to the human who initiated the task and to the agent session that explains the change. The company prefers built-in GitHub controls and auditability over creating a separate tracking system unless a requirement cannot be met through GitHub-native artifacts.

Northstar is documenting how engineering leads should supervise Copilot cloud agent work without creating a separate tracking system. Match each supervision need to the best GitHub-native mechanism.

The case study requires inspectable GitHub artifacts, the ability to steer active sessions, and traceability from agent-authored commits to the initiating human and session context.

Correct matches:

A lead wants to inspect why an active session is changing specific files.

→ **Open the session log and overview from the agents panel or agents page**

A release manager wants attribution from an agent commit to the initiating human.

→ **Use the Copilot-authored commit metadata, co-author, signature, and session-log link**

A developer wants to redirect an active session without discarding current progress.

→ **Send a follow-up steering prompt in the active agent session**

Choices: Open the session log and overview from the agents panel or agents page, Use the Copilot-authored commit metadata, co-author, signature, and session-log link, Send a follow-up steering prompt in the active agent session, Export local chat transcripts into a shared spreadsheet, Require a new repository issue for every agent tool call, Move the custom agent profile from `.github/agents` to `agents`

 Why the correct answer is right

Session logs and overviews expose the agent's progress and approach. Copilot cloud agent commits are authored by Copilot, include the initiating human as co-author, are signed, and link to session logs. Active sessions can be steered with follow-up prompts rather than stopped and restarted.

 For GH-600 observability questions, choose the GitHub-native artifact that answers the supervision need directly: session logs for reasoning, commit metadata for attribution, and steering prompts for course correction.

observability

traceability

session-steering

github-system-of-record

 Case study — Case Study: Meridian Grocery Pricing Agent Evaluation Program

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Meridian Grocery operates a regional online grocery platform. The engineering team is piloting GitHub Copilot cloud agent for small pricing-service changes, such as promotion rule updates, tax-calculation fixes, and checkout-message corrections. The organization wants to evaluate agent quality using evidence from GitHub pull requests, session logs, automated scans, workflow artifacts, and human review outcomes.

Existing Environment

The pricing-service repository is hosted on GitHub Enterprise Cloud and uses Copilot Business. Copilot cloud agent is enabled for the repository. The repository has unit tests, integration tests, linting, CodeQL, secret scanning, and dependency review checks. Copilot Memory is enabled for the organization. Repository owners periodically review stored memories, but no one has yet established a formal memory-curation process.

Business Requirements

The pricing team wants agent-authored pull requests to preserve checkout correctness, reduce reviewer rework, and avoid introducing security or dependency risk. The team wants metrics that distinguish between fast but low-quality output and changes that satisfy the original development intent. Evaluation results must be actionable enough to tune agent instructions, memory, workflow expectations, or tool access.

Technical Requirements

Each Copilot task should produce a branch and pull request that are traceable to the associated agent session. Evaluation must compare the agent's stated approach with the final diff and workflow artifacts. The team tracks test pass rate, coverage delta, CodeQL findings, secret scanning alerts, dependency-review findings, lint error count, review iterations, and session-log tool-call patterns. The team also stores qualitative reviewer tags, including 'wrong component,' 'unrelated churn,' 'missing test,' 'stale assumption,' and 'tool sequence issue.'

Security and Compliance Requirements

Security findings introduced by agent-authored changes must be visible before a pull request is considered ready for merge. The team must preserve enough telemetry to reconstruct why Copilot made a change. Tuning changes should address the observed root cause rather than masking symptoms. The team prefers preventive configuration changes, such as tighter instructions, memory cleanup, and narrower tool usage, over adding manual review effort for every low-risk task.

Meridian Grocery finds that several Copilot-authored pricing pull requests were opened quickly but later required rework for missing tests, dependency findings, and reviewer concerns. Which evaluation approach best aligns success criteria with the pricing-service development intent?

The platform team must define a reusable evaluation scorecard for future Copilot cloud agent tasks in the pricing-service repository.

- A** Use mean cycle time, token consumption, and developer satisfaction after merge as primary metrics; review scans only when users complain.
- B** Score success by whether Copilot opened a PR and completed within SLA; treat later defects as product backlog.
- C** Measure only model latency and prompt length, because repository checks and reviewer comments reflect downstream process issues.
- D** Track tests, coverage delta, scan findings, reviewer rework, and session-log tool counts against each pricing-change intent. ✓

✓ **Why the correct answer is right**

The correct approach combines quantitative signals, qualitative review evidence, and automated security signals, then ties them back to the task intent. GitHub documents that Copilot session logs show the tools used to understand, change, and validate work, and GitHub's GH-600 study guide explicitly includes evaluation signals, automated scanning tools, logs, traces, outputs, and workflow artifacts in this domain.

✗ **Why the other options are wrong**

- **A:** Cycle time and satisfaction are useful secondary signals, but delaying scan review until complaints appear misses preventive evaluation.
- **B:** Opening a PR quickly is not enough evidence that the change met pricing correctness, test, and security expectations.
- **C:** Latency and prompt length do not evaluate whether the agent produced correct, secure, reviewable repository changes.

💡 For GH-600 evaluation questions, prefer scorecards that combine outcome quality, scan results, artifacts, and trace evidence. Speed-only metrics are usually incomplete.

evaluation-signals

automated-scanning

success-criteria

 Case study — Case Study: Meridian Grocery Pricing Agent Evaluation Program

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Meridian Grocery operates a regional online grocery platform. The engineering team is piloting GitHub Copilot cloud agent for small pricing-service changes, such as promotion rule updates, tax-calculation fixes, and checkout-message corrections. The organization wants to evaluate agent quality using evidence from GitHub pull requests, session logs, automated scans, workflow artifacts, and human review outcomes.

Existing Environment

The pricing-service repository is hosted on GitHub Enterprise Cloud and uses Copilot Business. Copilot cloud agent is enabled for the repository. The repository has unit tests, integration tests, linting, CodeQL, secret scanning, and dependency review checks. Copilot Memory is enabled for the organization. Repository owners periodically review stored memories, but no one has yet established a formal memory-curation process.

Business Requirements

The pricing team wants agent-authored pull requests to preserve checkout correctness, reduce reviewer rework, and avoid introducing security or dependency risk. The team wants metrics that distinguish between fast but low-quality output and changes that satisfy the original development intent. Evaluation results must be actionable enough to tune agent instructions, memory, workflow expectations, or tool access.

Technical Requirements

Each Copilot task should produce a branch and pull request that are traceable to the associated agent session. Evaluation must compare the agent's stated approach with the final diff and workflow artifacts. The team tracks test pass rate, coverage delta, CodeQL findings, secret scanning alerts, dependency-review findings, lint error count, review iterations, and session-log tool-call patterns. The team also stores qualitative reviewer tags, including 'wrong component,' 'unrelated churn,' 'missing test,' 'stale assumption,' and 'tool sequence issue.'

Security and Compliance Requirements

Security findings introduced by agent-authored changes must be visible before a pull request is considered ready for merge. The team must preserve enough telemetry to reconstruct why Copilot made a change. Tuning changes should address the observed root cause rather than masking symptoms. The team prefers preventive configuration changes, such as tighter instructions, memory cleanup, and narrower tool usage, over adding manual review effort for every low-risk task.

Meridian Grocery reviews three failed Copilot sessions and wants to classify the primary root cause before tuning anything. Match each observed failure signal to the most defensible root-cause classification.

The review team has access to the session log, the first plan artifact, the final pull-request diff, and workflow artifacts for each failed session.

Correct matches:

The plan chose checkout-ui even though the issue and changed tests referenced pricing-service; every commit followed that wrong component choice.	→ Reasoning error
The trace shows `npm audit fix --force` ran before tests, upgrading unrelated packages and causing dependency-review findings.	→ Tool misuse
Integration tests failed because PAYMENT_SIM_URL was unavailable in the agent environment; the same commit passed in a maintainer environment.	→ Context or environment issue

Choices: Context or environment issue, Prompt or instruction issue, Reasoning error, Governance approval issue, Tool misuse

✔ Why the correct answer is right

The wrong component choice is a reasoning failure because the agent's plan diverged from the issue intent. The forced audit command is tool misuse because the tool choice and ordering caused unrelated changes. The unavailable PAYMENT_SIM_URL is a context or environment issue because the failure depends on the agent's runtime environment rather than the code diff alone.

💡 On GH-600 failure-analysis items, classify from evidence: compare plan to task for reasoning errors, inspect tool traces for misuse, and use workflow artifacts for environment gaps.

- failure-analysis
- root-cause-classification
- session-logs

 Case study — Case Study: Meridian Grocery Pricing Agent Evaluation Program

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Meridian Grocery operates a regional online grocery platform. The engineering team is piloting GitHub Copilot cloud agent for small pricing-service changes, such as promotion rule updates, tax-calculation fixes, and checkout-message corrections. The organization wants to evaluate agent quality using evidence from GitHub pull requests, session logs, automated scans, workflow artifacts, and human review outcomes.

Existing Environment

The pricing-service repository is hosted on GitHub Enterprise Cloud and uses Copilot Business. Copilot cloud agent is enabled for the repository. The repository has unit tests, integration tests, linting, CodeQL, secret scanning, and dependency review checks. Copilot Memory is enabled for the organization. Repository owners periodically review stored memories, but no one has yet established a formal memory-curation process.

Business Requirements

The pricing team wants agent-authored pull requests to preserve checkout correctness, reduce reviewer rework, and avoid introducing security or dependency risk. The team wants metrics that distinguish between fast but low-quality output and changes that satisfy the original development intent. Evaluation results must be actionable enough to tune agent instructions, memory, workflow expectations, or tool access.

Technical Requirements

Each Copilot task should produce a branch and pull request that are traceable to the associated agent session. Evaluation must compare the agent's stated approach with the final diff and workflow artifacts. The team tracks test pass rate, coverage delta, CodeQL findings, secret scanning alerts, dependency-review findings, lint error count, review iterations, and session-log tool-call patterns. The team also stores qualitative reviewer tags, including 'wrong component,' 'unrelated churn,' 'missing test,' 'stale assumption,' and 'tool sequence issue.'

Security and Compliance Requirements

Security findings introduced by agent-authored changes must be visible before a pull request is considered ready for merge. The team must preserve enough telemetry to reconstruct why Copilot made a change. Tuning changes should address the observed root cause rather than masking symptoms. The team prefers preventive configuration changes, such as tighter instructions, memory cleanup, and narrower tool usage, over adding manual review effort for every low-risk task.

Meridian Grocery's latest evaluation report shows that Copilot repeatedly uses a stale repository fact, edits generated lockfiles during unrelated pricing changes, and invokes broad shell commands where read/search would be safer. Select THREE tuning actions the team should take.

The team wants to reduce repeat failures without disabling Copilot cloud agent for low-risk pricing tasks.

 Evaluation excerpt

Recurring signals:

- Stored memory: "Pricing rules are defined in src/rules/*.json"
- Current architecture: pricing rules migrated to config/pricing/*.toml six weeks ago
- Session traces: frequent broad shell search and forced dependency updates
- Review pattern: unrelated lockfile churn and missing tests for TOML rule changes
- Security checks: dependency review sometimes flags newly introduced high-severity advisories

- A** Require senior engineers to manually rewrite every Copilot pricing PR before running repository checks.
- B** Increase the prompt token budget so Copilot can infer the migration from a larger context window.
- C** Add repository instructions that require pricing TOML tests and discourage unrelated generated-file changes. ✓
- D** Narrow the agent's expected tool pattern toward read/search before editing and verification before PR readiness. ✓
- E** Delete the stale Copilot Memory entry and add the current TOML pricing-location fact if needed. ✓
- F** Disable dependency review and secret scanning for agent-authored branches to reduce noisy evaluation failures.

 Why the correct answer is right

Deleting or correcting the stale memory addresses the repeated context error. Repository instructions can encode project-specific constraints, such as testing TOML pricing changes and avoiding unrelated generated-file churn. Tool-pattern guidance addresses the observed misuse by steering the agent toward repository inspection before editing and validation before PR readiness.

 Why the other options are wrong

- **F:** Disabling scans hides security and dependency signals that the evaluation program explicitly needs.
- **B:** A larger context window may not correct a validated stale memory or unsafe tool sequence.

- **A:** Manual rewrites add review burden without tuning the agent behavior that caused the failures.

💡 When telemetry shows repeat failures, tune the root cause: memory for stale facts, instructions for recurring constraints, and tool guidance for unsafe execution patterns.

tuning

memory-curation

tool-usage-refinement

Quick Answer Key

#	Type	Correct Answer
Q1	sequence order	B → D → F → C → E → A
Q2	multi select	C, A
Q3	single choice	C
Q4	single choice	C
Q5	single choice	A
Q6	single choice	B
Q7	code or config artifact	D
Q8	single choice	A
Q9	matching magnet	row1 → Let the agent create a draft pull request, then rely on human readiness, approval, and merge controls.; row2 → Use Copilot-authored signed commits, session-log links, and enterprise audit events for traceability.; row3 → Use environment protection with required reviewers on the staging deployment job, not on ordinary coding steps.
Q10	sequence order	C → D → B → A
Q11	single choice	C
Q12	multi select	C, B
Q13	sequence order	B → C → E → D → A
Q14	single choice	B
Q15	multi select	A, C
Q16	code or config	C

#	Type	Correct Answer
	artifact	
Q17	multi select	D, E
Q18	code or config artifact	B
Q19	multi select	F, C, E
Q20	single choice	C
Q21	code or config artifact	B
Q22	multi select	A, E
Q23	code or config artifact	B
Q24	multi select	B, C
Q25	single choice	C
Q26	code or config artifact	A
Q27	single choice	A
Q28	matching magnet	row1 → Create an Agents secret prefixed with COPILOT_MCP_ and reference that value from the MCP headers or env.; row2 → Configure an MCP registry URL and set MCP access restriction to registry servers for the organization.; row3 → Limit the custom agent tools to the needed GitHub MCP tool names or aliases instead of enabling all tools.; row4 → Replace the remote OAuth dependency with a supported PAT or non-OAuth authentication pattern for cloud agent.
Q29	code or config artifact	C
Q30	code or config	A

#	Type	Correct Answer
	artifact	
Q31	code or config artifact	B
Q32	single choice	B
Q33	multi select	B, E, F
Q34	single choice	A
Q35	multi select	D, C
Q36	code or config artifact	A
Q37	matching magnet	row1 → Repository-level Copilot Memory fact scoped to the same repository; row2 → User-level Copilot Memory preference scoped to the initiating user; row3 → External durable memory such as a committed state file or system of record; row4 → Short-term context window for the active session
Q38	matching magnet	row1 → Publishes a governed organization catalog readable by members without manual per-user sharing; row2 → Defines a reusable specialized agent profile that Copilot can load from the catalog; row3 → Prevents automatic context-based use while still allowing intentional human selection; row4 → Makes the agent produce a structured planning artifact suitable for review
Q39	matching magnet	row1 → Defines the task input contract before the agent starts work; row2 → Provides an inspectable planning artifact before implementation is accepted; row3 → Creates traceability from repository history back to the agent run; row4 → Acts as an evaluation gate inside the normal PR workflow
Q40	code or config artifact	D
Q41	multi select	D, B
Q42	single choice	A
Q43	multi select	B, E

#	Type	Correct Answer
Q44	code or config artifact	D
Q45	single choice	B
Q46	matching magnet	row1 → Rely on hidden-character filtering for the HTML comment, review the visible request, and retain the session record for audit.; row2 → Inspect the workflow changes before selecting Approve and run workflows; keep the approval gate enabled for agent PRs.; row3 → Create a narrowly scoped Agents secret for the registry and keep Actions secrets outside the agent environment.
Q47	single choice	B
Q48	multi select	F, D, E
Q49	multi select	D, E
Q50	single choice	D
Q51	code or config artifact	B
Q52	single choice	C
Q53	sequence order	B → D → C → E → A
Q54	sequence order	B → A → C → D
Q55	multi select	B, F
Q56	code or config artifact	A
Q57	single choice	B
Q58	multi select	D, C
Q59	matching magnet	row1 → Create an Agents secret or variable for the token so setup steps can consume it as an environment variable.; row2 → Add only the required host to

#	Type	Correct Answer
		the Copilot cloud agent firewall allowlist or approved network path.; row3 → Use an appropriate Windows runner strategy and implement runner-level network controls because the integrated firewall is not compatible there.; row4 → Rename the job to copilot-setup-steps in .github/workflows/copilot-setup-steps.yml on the default branch.
Q60	single choice	A
Q61	matching magnet	row1 → Configure an MCP registry URL and set MCP access to Registry only; row2 → Add the exact staging API URL to the cloud agent firewall custom allowlist; row3 → Store the token as an Agents secret prefixed COPILOT_MCP_ and reference it in MCP config
Q62	sequence order	C → A → D → B
Q63	multi select	B, A
Q64	single choice	C
Q65	matching magnet	row1 → Open the session log and overview from the agents panel or agents page; row2 → Use the Copilot-authored commit metadata, co-author, signature, and session-log link; row3 → Send a follow-up steering prompt in the active agent session
Q66	single choice	D
Q67	matching magnet	row1 → Reasoning error; row2 → Tool misuse; row3 → Context or environment issue
Q68	multi select	E, C, D